

CHAPTER 6



Formal Relational Query Languages

Practice Exercises

- 6.1 Write the following queries in relational algebra, using the university schema.
- Find the titles of courses in the Comp. Sci. department that have 3 credits.
 - Find the IDs of all students who were taught by an instructor named Einstein; make sure there are no duplicates in the result.
 - Find the highest salary of any instructor.
 - Find all instructors earning the highest salary (there may be more than one with the same salary).
 - Find the enrollment of each section that was offered in Autumn 2009.
 - Find the maximum enrollment, across all sections, in Autumn 2009.
 - Find the sections that had the maximum enrollment in Autumn 2009

Answer:

- $\Pi_{title}(\sigma_{dept_name = 'Comp. Sci' \wedge credits=3}(course))$
- $\Pi_{ID}(\sigma_{IID = 'Einstein'}(takes \bowtie \rho_{t1(IID, course_id, section_id, semester, year)} teaches))$
Assuming the set version of the relational algebra is used, there is no need to explicitly remove duplicates. If the multiset version is used, the grouping operator can be used without any aggregation to remove duplicates. For example given relation $r(A, B)$ possibly containing duplicates, $_{A,B}\mathcal{G}(r)$ would return a duplicate free version of the relation.
- $\mathcal{G}_{\max(salary)}(instructor)$

- d. $instructor \bowtie (\mathcal{G}_{\max(salary)} \text{ as } salary(instructor))$
 Note that the above query renames the maximum salary as salary, so the subsequent natural join outputs only instructors with that salary.
- e. $course_id, section_id \mathcal{G}_{\text{count}(*)} \text{ as } enrollment(\sigma_{year=2009 \wedge semester=Autumn}(takes))$
- f. $t1 \leftarrow course_id, section_id \mathcal{G}_{\text{count}(*)} \text{ as } enrollment(\sigma_{year=2009 \wedge semester=Autumn}(takes))$
 $result = \mathcal{G}_{\max(enrollment)}(t1)$
- g. $t2 \leftarrow \mathcal{G}_{\max(enrollment)} \text{ as } enrollment(t1)$
 where $t1$ is as defined in the previous part of the question.
 $result = t1 \bowtie t2$

6.2 Consider the relational database of Figure 6.22, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- Find the names of all employees who live in the same city and on the same street as do their managers.
- Find the names of all employees in this database who do not work for “First Bank Corporation”.
- Find the names of all employees who earn more than every employee of “Small Bank Corporation”.

Answer:

- $\Pi_{person_name} ((employee \bowtie manages) \bowtie (\sigma_{(manager_name = employee2.person_name \wedge employee.street = employee2.street \wedge employee.city = employee2.city)}(\rho_{employee2}(employee))))$
- The following solutions assume that all people work for exactly one company. If one allows people to appear in the database (e.g. in *employee*) but not appear in *works*, the problem is more complicated. We give solutions for this more realistic case later.
 $\Pi_{person_name} (\sigma_{company_name \neq \text{“First Bank Corporation”}}(works))$
 If people may not work for any company:
 $\Pi_{person_name}(employee) - \Pi_{person_name}(\sigma_{(company_name = \text{“First Bank Corporation”}}(works))$
- $\Pi_{person_name}(works) - (\Pi_{works.person_name}(works \bowtie (\sigma_{(works.salary \leq works2.salary \wedge works2.company_name = \text{“Small Bank Corporation”}}(\rho_{works2}(works))))$

6.3 The natural outer-join operations extend the natural-join operation so that tuples from the participating relations are not lost in the result of the join.

Describe how the theta-join operation can be extended so that tuples from the left, right, or both relations are not lost from the result of a theta join.

Answer:

- The left outer theta join of $r(R)$ and $s(S)$ ($r \bowtie_{\theta}^L s$) can be defined as $(r \bowtie_{\theta} s) \cup ((r - \Pi_R(r \bowtie_{\theta} s)) \times (null, null, \dots, null))$. The tuple of nulls is of size equal to the number of attributes in S .
- The right outer theta join of $r(R)$ and $s(S)$ ($r \bowtie_{\theta}^R s$) can be defined as $(r \bowtie_{\theta} s) \cup ((null, null, \dots, null) \times (s - \Pi_S(r \bowtie_{\theta} s)))$. The tuple of nulls is of size equal to the number of attributes in R .
- The full outer theta join of $r(R)$ and $s(S)$ ($r \bowtie_{\theta}^F s$) can be defined as $(r \bowtie_{\theta} s) \cup ((null, null, \dots, null) \times (s - \Pi_S(r \bowtie_{\theta} s))) \cup ((r - \Pi_R(r \bowtie_{\theta} s)) \times (null, null, \dots, null))$. The first tuple of nulls is of size equal to the number of attributes in R , and the second one is of size equal to the number of attributes in S .

6.4 (Division operation): The division operator of relational algebra, “ $\div$ ”, is defined as follows. Let $r(R)$ and $s(S)$ be relations, and let $S \subseteq R$; that is, every attribute of schema S is also in schema R . Then $r \div s$ is a relation on schema $R - S$ (that is, on the schema containing all attributes of schema R that are not in schema S). A tuple t is in $r \div s$ if and only if both of two conditions hold:

- t is in $\Pi_{R-S}(r)$
- For every tuple t_s in s , there is a tuple t_r in r satisfying both of the following:
 - $t_r[S] = t_s[S]$
 - $t_r[R - S] = t$

Given the above definition:

- Write a relational algebra expression using the division operator to find the IDs of all students who have taken all Comp. Sci. courses. (Hint: project *takes* to just ID and *course_id*, and generate the set of all Comp. Sci. *course_ids* using a select expression, before doing the division.)
- Show how to write the above query in relational algebra, without using division. (By doing so, you would have shown how to define the division operation using the other relational algebra operations.)

Answer:

- $\Pi_{ID}(\Pi_{ID, course_id}(takes) \div \Pi_{course_id}(\sigma_{dept_name='Comp. Sci'}(course)))$
- The required expression is as follows:

$$r \leftarrow \Pi_{ID, course_id}(takes)$$

$$s \leftarrow \Pi_{course_id}(\sigma_{dept_name='Comp. Sci'}(course))$$

$$\Pi_{ID}(takes) - \Pi_{ID}((\Pi_{ID}(takes) \times s) - r)$$

In general, let $r(R)$ and $s(S)$ be given, with $S \subseteq R$. Then we can express the division operation using basic relational algebra operations as follows:

$$r \div s = \Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

To see that this expression is true, we observe that $\Pi_{R-S}(r)$ gives us all tuples t that satisfy the first condition of the definition of division. The expression on the right side of the set difference operator

$$\Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

serves to eliminate those tuples that fail to satisfy the second condition of the definition of division. Let us see how it does so. Consider $\Pi_{R-S}(r) \times s$. This relation is on schema R , and pairs every tuple in $\Pi_{R-S}(r)$ with every tuple in s . The expression $\Pi_{R-S,S}(r)$ merely reorders the attributes of r .

Thus, $(\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r)$ gives us those pairs of tuples from $\Pi_{R-S}(r)$ and s that do not appear in r . If a tuple t_j is in

$$\Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

then there is some tuple t_s in s that does not combine with tuple t_j to form a tuple in r . Thus, t_j holds a value for attributes $R - S$ that does not appear in $r \div s$. It is these values that we eliminate from $\Pi_{R-S}(r)$.

6.5 Let the following relation schemas be given:

$$R = (A, B, C)$$

$$S = (D, E, F)$$

Let relations $r(R)$ and $s(S)$ be given. Give an expression in the tuple relational calculus that is equivalent to each of the following:

- $\Pi_A(r)$
- $\sigma_{B=17}(r)$
- $r \times s$
- $\Pi_{A,F}(\sigma_{C=D}(r \times s))$

Answer:

- $\{t \mid \exists q \in r (q[A] = t[A])\}$
- $\{t \mid t \in r \wedge t[B] = 17\}$
- $\{t \mid \exists p \in r \exists q \in s (t[A] = p[A] \wedge t[B] = p[B] \wedge t[C] = p[C] \wedge t[D] = q[D] \wedge t[E] = q[E] \wedge t[F] = q[F])\}$

- d. $\{t \mid \exists p \in r \exists q \in s (t[A] = p[A] \wedge t[F] = q[F] \wedge p[C] = q[D])\}$
- 6.6 Let $R = (A, B, C)$, and let r_1 and r_2 both be relations on schema R . Give an expression in the domain relational calculus that is equivalent to each of the following:
- $\Pi_A(r_1)$
 - $\sigma_{B=17}(r_1)$
 - $r_1 \cup r_2$
 - $r_1 \cap r_2$
 - $r_1 - r_2$
 - $\Pi_{A,B}(r_1) \bowtie \Pi_{B,C}(r_2)$

Answer:

- $\{ \langle t \rangle \mid \exists p, q (\langle t, p, q \rangle \in r_1) \}$
 - $\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \wedge b = 17 \}$
 - $\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \vee \langle a, b, c \rangle \in r_2 \}$
 - $\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \wedge \langle a, b, c \rangle \in r_2 \}$
 - $\{ \langle a, b, c \rangle \mid \langle a, b, c \rangle \in r_1 \wedge \langle a, b, c \rangle \notin r_2 \}$
 - $\{ \langle a, b, c \rangle \mid \exists p, q (\langle a, b, p \rangle \in r_1 \wedge \langle q, b, c \rangle \in r_2) \}$
- 6.7 Let $R = (A, B)$ and $S = (A, C)$, and let $r(R)$ and $s(S)$ be relations. Write expressions in relational algebra for each of the following queries:
- $\{ \langle a \rangle \mid \exists b (\langle a, b \rangle \in r \wedge b = 7) \}$
 - $\{ \langle a, b, c \rangle \mid \langle a, b \rangle \in r \wedge \langle a, c \rangle \in s \}$
 - $\{ \langle a \rangle \mid \exists c (\langle a, c \rangle \in s \wedge \exists b_1, b_2 (\langle a, b_1 \rangle \in r \wedge \langle c, b_2 \rangle \in r \wedge b_1 > b_2)) \}$

Answer:

- $\Pi_A(\sigma_{B=7}(r))$
 - $r \bowtie s$
 - $\Pi_A(s \bowtie (\Pi_{r,A}(\sigma_{r.b=d.b}(r \times \rho_d(r)))))$
- 6.8 Consider the relational database of Figure 6.22 where the primary keys are underlined. Give an expression in tuple relational calculus for each of the following queries:
- Find all employees who work directly for “Jones.”
 - Find all cities of residence of all employees who work directly for “Jones.”

- c. Find the name of the manager of the manager of “Jones.”
- d. Find those employees who earn more than all employees living in the city “Mumbai.”

Answer:

a.

$$\{t \mid \exists m \in \text{manages} (t[\text{person_name}] = m[\text{person_name}] \wedge m[\text{manager_name}] = \text{'Jones'})\}$$

b.

$$\{t \mid \exists m \in \text{manages} \exists e \in \text{employee} (e[\text{person_name}] = m[\text{person_name}] \wedge m[\text{manager_name}] = \text{'Jones'} \wedge t[\text{city}] = e[\text{city}])\}$$

c.

$$\{t \mid \exists m1 \in \text{manages} \exists m2 \in \text{manages} (m1[\text{manager_name}] = m2[\text{person_name}] \wedge m1[\text{person_name}] = \text{'Jones'} \wedge t[\text{manager_name}] = m2[\text{manager_name}])\}$$

d.

$$\{t \mid \exists w1 \in \text{works} \neg \exists w2 \in \text{works} (w1[\text{salary}] < w2[\text{salary}] \wedge \exists e2 \in \text{employee} (w2[\text{person_name}] = e2[\text{person_name}] \wedge e2[\text{city}] = \text{'Mumbai'}))\}$$

6.9 Describe how to translate join expressions in SQL to relational algebra.

Answer: A query of the form

```
select A1, A2, ..., An
from R1, R2, ..., Rm
where P
```

can be translated into relational algebra as follows:

$$\Pi_{A1, A2, \dots, An}(\sigma_P(R1 \times R2 \times \dots \times Rm))$$

An SQL join expression of the form

R1 natural join R2

can be written as $R1 \bowtie R2$.

An SQL join expression of the form

R1 join R2 on (P)

can be written as $R1 \bowtie_P R2$.

An SQL join expression of the form

$$R1 \text{ join } R2 \text{ using } (A1, A2, \dots, An)$$

can be written as $\Pi_S(R1 \bowtie_{R1.A1=R2.A1 \wedge R1.A2=R2.A2 \wedge \dots R1.An=R2.An} R2)$ where S is $A1, A2, \dots, An$ followed by all attributes of $R1$ other than $R1.A1, R1.A2, \dots, R1.An$, followed by all attributes of $R2$ other than $R2.A1, R2.A2, \dots, R2.An$,

The outer join versions of the SQL join expressions can be similarly written by using $\bowtie^L$, $\bowtie^R$ and $\bowtie^F$ in place of $\bowtie$.¹

The most direct way to handle subqueries is to extend the relational algebra. To handle where clause subqueries, we need to allow selection predicates to contain nested relational algebra expressions, which can reference correlation attributes from outer level relations. Scalar subqueries can be similarly translated by allowing nested relational algebra expressions to appear in scalar expressions. An alternative approach to handling such subqueries used in some database systems, such as Microsoft SQL Server, introduces a new relational algebra operator called the Apply operator; see Chapter 30, page 1230-1231 for details. Without such extensions, translating subqueries into standard relational algebra can be rather complicated.

¹The case of outer joins with the **using** clause is a little more complicated; with a right outer join it is possible that $R1.A1$ is null, but $R2.A1$ is not, and the output should contain the non-null value. The SQL **coalesce** function can be used, replacing S by **coalesce**($R1.A1, R2.A1$), **coalesce**($R1.A2, R2.A2$), ... **coalesce**($R1.An, R2.An$), followed by the other attributes of $R1$ and $R2$.

试用水印

CHAPTER 6



Formal Relational Query Languages

In this chapter we study three additional formal relational languages. Relational Algebra, tuple relational calculus and domain relational calculus.

Of these three formal languages, we suggest placing an emphasis on relational algebra, which is used extensively in the chapters on query processing and optimization, as well as in several other chapters. The relational calculi generally do not merit as much emphasis.

Our notation for the tuple relational calculus makes it easy to present the concept of a safe query. The concept of safety for the domain relational calculus, though identical to that for the tuple calculus, is much more cumbersome notationally and requires careful presentation. This consideration may suggest placing somewhat less emphasis on the domain calculus for classes not focusing on database theory.

Exercises

- 6.10** Write the following queries in relational algebra, using the university schema.
- Find the names of all students who have taken at least one Comp. Sci. course.
 - Find the IDs and names of all students who have not taken any course offering before Spring 2009.
 - For each department, find the maximum salary of instructors in that department. You may assume that every department has at least one instructor.
 - Find the lowest, across all departments, of the per-department maximum salary computed by the preceding query.

Answer:

$employee(\underline{person_name}, street, city)$
 $works(\underline{person_name}, company_name, salary)$
 $company(\underline{company_name}, city)$
 $manages(\underline{person_name}, manager_name)$

Figure 6.22 Relational database for Exercises 6.2, 6.8, 6.11, 6.13, and 6.15

- a. $\Pi_{name}(student \bowtie takes \bowtie \Pi_{course_id}(\sigma_{dept_name = 'Comp.Sci.'}(course)))$
 Note that if we join *student*, *takes*, and *course*, only students from the Comp. Sci. department would be present in the result; students from other departments would be eliminated even if they had taken a Comp. Sci. course since the attribute *dept_name* appears in both *student* and *course*.
- b. $\Pi_{ID,name}(student) - \Pi_{ID,name}(\sigma_{year < 2009}(student \bowtie takes))$ Note that Spring is the first semester of the year, so we do not need to perform a comparison on *semester*.
- c. $dept_name \mathcal{G}_{\max(salary)}(instructor)$
- d. $\mathcal{G}_{\min(maxsal)}(dept_name \mathcal{G}_{\max(salary)} \text{ as } maxsal(instructor))$

6.11 Consider the relational database of Figure 6.22, where the primary keys are underlined. Give an expression in the relational algebra to express each of the following queries:

- a. Find the names of all employees who work for “First Bank Corporation”.
- b. Find the names and cities of residence of all employees who work for “First Bank Corporation”.
- c. Find the names, street addresses, and cities of residence of all employees who work for “First Bank Corporation” and earn more than \$10,000.
- d. Find the names of all employees in this database who live in the same city as the company for which they work.
- e. Assume the companies may be located in several cities. Find all companies located in every city in which “Small Bank Corporation” is located.

Answer:

- a. $\Pi_{person_name}(\sigma_{company_name = \text{“First Bank Corporation”}}(works))$
- b. $\Pi_{person_name, city}(employee \bowtie (\sigma_{company_name = \text{“First Bank Corporation”}}(works)))$

- c. $\Pi_{person_name, street, city}$
 $(\sigma_{(company_name = \text{"First Bank Corporation"} \wedge salary > 10000)}$
 $works \bowtie employee)$
- d. $\Pi_{person_name} (employee \bowtie works \bowtie company)$
- e. Note: Small Bank Corporation will be included in each answer.
 $\Pi_{company_name} (company \div$
 $(\Pi_{city} (\sigma_{company_name = \text{"Small Bank Corporation"} (company))))$

6.12 Using the university example, write relational-algebra queries to find the course sections taught by more than one instructor in the following ways:

- a. Using an aggregate function.
- b. Without using any aggregate functions.

Answer:

- a. $\sigma_{instruct > 1} (course_id, section_id, year, semester \mathcal{G}_{count(*)} \text{ as } instruct (teaches))$
- b. $\Pi_{course_id, section_id, year, semester} (\sigma_{ID < > ID_2} (takes \bowtie$
 $\rho_{takes1}(ID_2, course_id, section_id, year, semester) (takes)))$

6.13 Consider the relational database of Figure 6.22. Give a relational-algebra expression for each of the following queries:

- a. Find the company with the most employees.
- b. Find the company with the smallest payroll.
- c. Find those companies whose employees earn a higher salary, on average, than the average salary at First Bank Corporation.

Answer:

- a. $t_1 \leftarrow company_name \mathcal{G}_{count-distinct}(person_name)(works)$
 $t_2 \leftarrow \mathcal{G}_{max}(num_employees)(\rho_{company_strength}(company_name, num_employees)(t_1))$
 $\Pi_{company_name} (\rho_{t_3}(company_name, num_employees)(t_1) \bowtie \rho_{t_4}(num_employees)(t_2))$
- b. $t_1 \leftarrow company_name \mathcal{G}_{sum}(salary)(works)$
 $t_2 \leftarrow \mathcal{G}_{min}(payroll)(\rho_{company_payroll}(company_name, payroll)(t_1))$
 $\Pi_{company_name} (\rho_{t_3}(company_name, payroll)(t_1) \bowtie \rho_{t_4}(payroll)(t_2))$
- c. $t_1 \leftarrow company_name \mathcal{G}_{avg}(salary)(works)$
 $t_2 \leftarrow \sigma_{company_name = \text{"First Bank Corporation"}}(t_1)$
 $\Pi_{t_3.company_name} ((\rho_{t_3}(company_name, avg_salary)(t_1))$
 $\bowtie_{t_3.avg_salary > first_bank.avg_salary} (\rho_{first_bank}(company_name, avg_salary)(t_2)))$

6.14 Consider the following relational schema for a library:

$member(\underline{memb_no}, name, dob)$
 $books(\underline{isbn}, title, authors, publisher)$
 $borrowed(\underline{memb_no}, \underline{isbn}, date)$

Write the following queries in relational algebra.

- Find the names of members who have borrowed any book published by “McGraw-Hill”.
- Find the name of members who have borrowed all books published by “McGraw-Hill”.
- Find the name and membership number of members who have borrowed more than five different books published by “McGraw-Hill”.
- For each publisher, find the name and membership number of members who have borrowed more than five books of that publisher.
- Find the average number of books borrowed per member. Take into account that if an member does not borrow any books, then that member does not appear in the *borrowed* relation at all.

Answer:

- $$t_1 \leftarrow \Pi_{isbn}(\sigma_{publisher="McGraw-Hill"}(books))$$

$$\Pi_{name}((member \bowtie borrowed) \bowtie t_1)$$
- $$t_1 \leftarrow \Pi_{isbn}(\sigma_{publisher="McGraw-Hill"}(books))$$

$$\Pi_{name, isbn}(member \bowtie borrowed) \div t_1$$
- $$t_1 \leftarrow member \bowtie borrowed \bowtie (\sigma_{publisher="McGraw-Hill"}(books))$$

$$\Pi_{name}(\sigma_{countisbn > 5}((memb_no \text{ } G_{count-distinct(isbn) \text{ as } countisbn}(t_1))))$$
- $$t_1 \leftarrow member \bowtie borrowed \bowtie books$$

$$\Pi_{publisher, name}(\sigma_{countisbn > 5}((publisher, memb_no \text{ } G_{count-distinct(isbn) \text{ as } countisbn}(t_1))))$$

6.15 Consider the employee database of Figure 6.22. Give expressions in tuple relational calculus and domain relational calculus for each of the following queries:

- Find the names of all employees who work for “First Bank Corporation”.
- Find the names and cities of residence of all employees who work for “First Bank Corporation”.
- Find the names, street addresses, and cities of residence of all employees who work for “First Bank Corporation” and earn more than \$10,000.

- d. Find all employees who live in the same city as that in which the company for which they work is located.
- e. Find all employees who live in the same city and on the same street as their managers.
- f. Find all employees in the database who do not work for “First Bank Corporation”.
- g. Find all employees who earn more than every employee of “Small Bank Corporation”.
- h. Assume that the companies may be located in several cities. Find all companies located in every city in which “Small Bank Corporation” is located.

Answer:

- a. Find the names of all employees who work for First Bank Corporation:
 - i. $\{t \mid \exists s \in works (t[person_name] = s[person_name] \wedge s[company_name] = \text{“First Bank Corporation”})\}$
 - ii. $\{ \langle p \rangle \mid \exists c, s (\langle p, c, s \rangle \in works \wedge c = \text{“First Bank Corporation”}) \}$
- b. Find the names and cities of residence of all employees who work for First Bank Corporation:
 - i. $\{t \mid \exists r \in employee \exists s \in works (t[person_name] = r[person_name] \wedge t[city] = r[city] \wedge r[person_name] = s[person_name] \wedge s[company_name] = \text{“First Bank Corporation”})\}$
 - ii. $\{ \langle p, c \rangle \mid \exists co, sa, st (\langle p, co, sa \rangle \in works \wedge \langle p, st, c \rangle \in employee \wedge co = \text{“First Bank Corporation”}) \}$
- c. Find the names, street address, and cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000 per annum:
 - i. $\{t \mid t \in employee \wedge (\exists s \in works (s[person_name] = t[person_name] \wedge s[company_name] = \text{“First Bank Corporation”} \wedge s[salary] > 10000))\}$
 - ii. $\{ \langle p, s, c \rangle \mid \langle p, s, c \rangle \in employee \wedge \exists co, sa (\langle p, co, sa \rangle \in works \wedge co = \text{“First Bank Corporation”} \wedge sa > 10000) \}$
- d. Find the names of all employees in this database who live in the same city as the company for which they work:

- i. $\{t \mid \exists e \in \text{employee} \exists w \in \text{works} \exists c \in \text{company} \\ (t[\text{person_name}] = e[\text{person_name}] \\ \wedge e[\text{person_name}] = w[\text{person_name}] \\ \wedge w[\text{company_name}] = c[\text{company_name}] \wedge e[\text{city}] = \\ c[\text{city}])\}$
- ii. $\{ \langle p \rangle \mid \exists st, c, co, sa (\langle p, st, c \rangle \in \text{employee} \\ \wedge \langle p, co, sa \rangle \in \text{works} \wedge \langle co, c \rangle \in \text{company})\}$
- e. Find the names of all employees who live in the same city and on the same street as do their managers:
 - i. $\{t \mid \exists l \in \text{employee} \exists m \in \text{manages} \exists r \in \text{employee} \\ (l[\text{person_name}] = m[\text{person_name}] \wedge m[\text{manager_name}] = \\ r[\text{person_name}] \\ \wedge l[\text{street}] = r[\text{street}] \wedge l[\text{city}] = r[\text{city}] \wedge t[\text{person_name}] = \\ l[\text{person_name}])\}$
 - ii. $\{ \langle t \rangle \mid \exists s, c, m (\langle t, s, c \rangle \in \text{employee} \wedge \langle t, m \rangle \in \\ \text{manages} \wedge \langle m, s, c \rangle \in \text{employee})\}$
- f. Find the names of all employees in this database who do not work for First Bank Corporation:
 If one allows people to appear in the database (e.g. in *employee*) but not appear in *works*, the problem is more complicated. We give solutions for this more realistic case later.
 - i. $\{t \mid \exists w \in \text{works} (w[\text{company_name}] \neq \text{"First Bank Corporation"} \\ \wedge t[\text{person_name}] = w[\text{person_name}])\}$
 - ii. $\{ \langle p \rangle \mid \exists c, s (\langle p, c, s \rangle \in \text{works} \wedge c \neq \text{"First Bank Corporation"})\}$
 If people may not work for any company:
 - i. $\{t \mid \exists e \in \text{employee} (t[\text{person_name}] = e[\text{person_name}] \wedge \neg \exists w \in \\ \text{works} \\ (w[\text{company_name}] = \text{"First Bank Corporation"} \\ \wedge w[\text{person_name}] = t[\text{person_name}]))\}$
 - ii. $\{ \langle p \rangle \mid \exists s, c (\langle p, s, c \rangle \in \text{employee}) \wedge \neg \exists x, y \\ (y = \text{"First Bank Corporation"} \wedge \langle p, y, x \rangle \in \text{works})\}$
- g. Find the names of all employees who earn more than every employee of Small Bank Corporation:
 - i. $\{t \mid \exists w \in \text{works} (t[\text{person_name}] = w[\text{person_name}] \wedge \forall s \in \\ \text{works} \\ (s[\text{company_name}] = \text{"Small Bank Corporation"} \Rightarrow w[\text{salary}] > \\ s[\text{salary}]))\}$

- ii. $\{ \langle p \rangle \mid \exists c, s (\langle p, c, s \rangle \in \text{works} \wedge \forall p_2, c_2, s_2 (\langle p_2, c_2, s_2 \rangle \notin \text{works} \vee c_2 \neq \text{"Small Bank Corporation"} \vee s > s_2)) \}$
- h. Assume the companies may be located in several cities. Find all companies located in every city in which Small Bank Corporation is located.
Note: Small Bank Corporation will be included in each answer.
- i. $\{ t \mid \forall s \in \text{company} (s[\text{company_name}] = \text{"Small Bank Corporation"} \Rightarrow \exists r \in \text{company} (t[\text{company_name}] = r[\text{company_name}] \wedge r[\text{city}] = s[\text{city}])) \}$
- ii. $\{ \langle co \rangle \mid \forall co_2, ci_2 (\langle co_2, ci_2 \rangle \notin \text{company} \vee co_2 \neq \text{"Small Bank Corporation"} \vee \langle co, ci_2 \rangle \in \text{company}) \}$
- 6.16 Let $R = (A, B)$ and $S = (A, C)$, and let $r(R)$ and $s(S)$ be relations. Write relational-algebra expressions equivalent to the following domain-relational-calculus expressions:
- a. $\{ \langle a \rangle \mid \exists b (\langle a, b \rangle \in r \wedge b = 17) \}$
- b. $\{ \langle a, b, c \rangle \mid \langle a, b \rangle \in r \wedge \langle a, c \rangle \in s \}$
- c. $\{ \langle a \rangle \mid \exists b (\langle a, b \rangle \in r) \vee \forall c (\exists d (\langle d, c \rangle \in s) \Rightarrow \langle a, c \rangle \in s) \}$
- d. $\{ \langle a \rangle \mid \exists c (\langle a, c \rangle \in s \wedge \exists b_1, b_2 (\langle a, b_1 \rangle \in r \wedge \langle c, b_2 \rangle \in r \wedge b_1 > b_2)) \}$

Answer:

- a. $\Pi_A (\sigma_{B=17} (r))$
- b. $r \bowtie s$
- c. $\Pi_A(r) \cup (r \div \sigma_B(\Pi_C(s)))$
- d. $\Pi_{r,A} ((r \bowtie s) \bowtie_{c=r2.A \wedge r.B > r2.B} (\rho_{r2}(r)))$
It is interesting to note that (d) is an abstraction of the notorious query "Find all employees who earn more than their manager." Let $R = (emp, sal)$, $S = (emp, mgr)$ to observe this.
- 6.17 Repeat Exercise 6.16, writing SQL queries instead of relational-algebra expressions.

Answer:

- a. **select** a
from r
where $b = 17$

- b. **select** a, b, c
from r, s
where $r.a = s.a$
- c. (**select** a
from r)
union
(**select** a
from s)
- d. **select** a
from r **as** $r1, r$ **as** $r2, s$
where $r1.a = s.a$ **and** $r2.a = s.c$ **and** $r1.b > r2.b$

6.18 Let $R = (A, B)$ and $S = (A, C)$, and let $r(R)$ and $s(S)$ be relations. Using the special constant *null*, write tuple-relational-calculus expressions equivalent to each of the following:

- a. $r \bowtie s$
- b. $r \supset \bowtie s$
- c. $r \Join s$

Answer:

- a. $\{t \mid \exists r \in R \exists s \in S (r[A] = s[A] \wedge t[A] = r[A] \wedge t[B] = r[B] \wedge t[C] = s[C]) \vee \exists s \in S (\neg \exists r \in R (r[A] = s[A]) \wedge t[A] = s[A] \wedge t[C] = s[C] \wedge t[B] = \text{null})\}$
- b. $\{t \mid \exists r \in R \exists s \in S (r[A] = s[A] \wedge t[A] = r[A] \wedge t[B] = r[B] \wedge t[C] = s[C]) \vee \exists r \in R (\neg \exists s \in S (r[A] = s[A]) \wedge t[A] = r[A] \wedge t[B] = r[B] \wedge t[C] = \text{null}) \vee \exists s \in S (\neg \exists r \in R (r[A] = s[A]) \wedge t[A] = s[A] \wedge t[C] = s[C] \wedge t[B] = \text{null})\}$
- c. $\{t \mid \exists r \in R \exists s \in S (r[A] = s[A] \wedge t[A] = r[A] \wedge t[B] = r[B] \wedge t[C] = s[C]) \vee \exists r \in R (\neg \exists s \in S (r[A] = s[A]) \wedge t[A] = r[A] \wedge t[B] = r[B] \wedge t[C] = \text{null})\}$

6.19 Give a tuple-relational-calculus expression to find the maximum value in relation $r(A)$.

Answer: $\{ \langle a \rangle \mid \langle a \rangle \in r \wedge \forall \langle b \rangle \in R \ a \geq b \}$

CHAPTER 7



Database Design and the E-R Model

Practice Exercises

- 7.1 **Answer:** The E-R diagram is shown in Figure 7.1. Payments are modeled as weak entities since they are related to a specific policy. Note that the participation of accident in the relationship *participated* is not total, since it is possible that there is an accident report where the participating car is unknown.

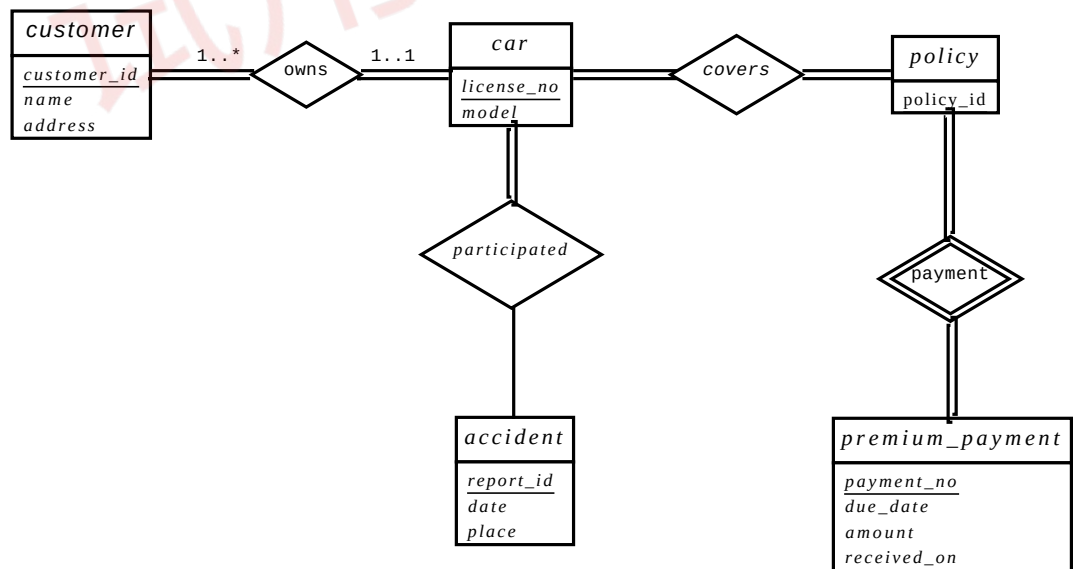


Figure 7.1 E-R diagram for a car insurance company.

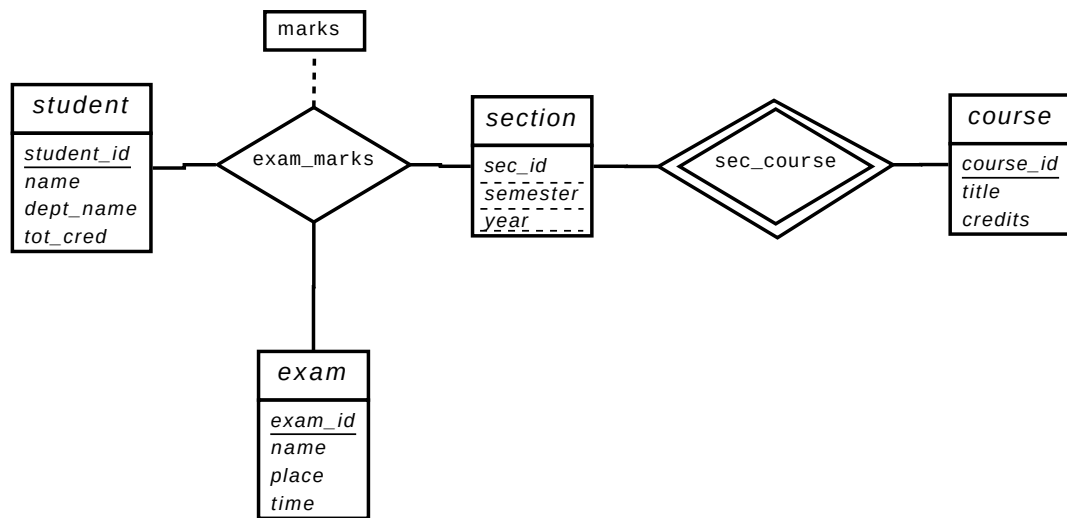


Figure 7.2 E-R diagram for marks database.

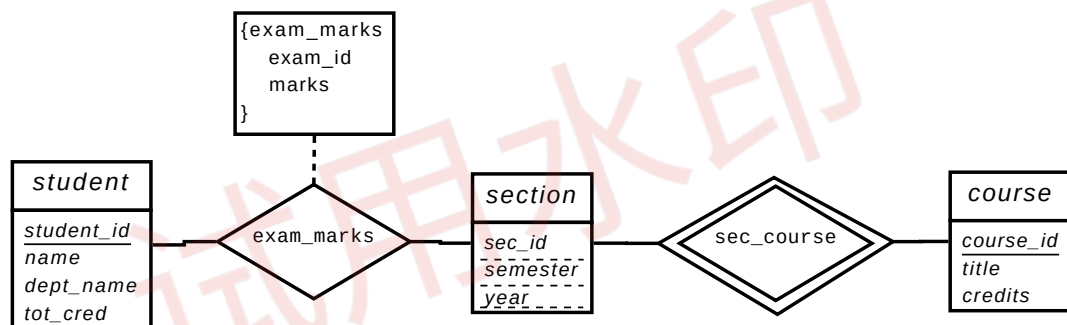


Figure 7.3 Another E-R diagram for marks database.

7.2 **Answer:** Note: the name of the relationship "course offering" needs to be changed to "section".

- The E-R diagram is shown in Figure 7.2. Note that an alternative is to model examinations as weak entities related to a section, rather than as a strong entity. The marks relationship would then be a binary relationship between *student* and *exam*, without directly involving *section*.
- The E-R diagram is shown in Figure 7.3. Note that here we have not modeled the name, place and time of the exam as part of the relationship attributes. Doing so would result in duplication of the information, once per student, and we would not be able to record this information without an associated student. If we wish to represent this information, we would need to retain a separate entity corresponding to each exam.

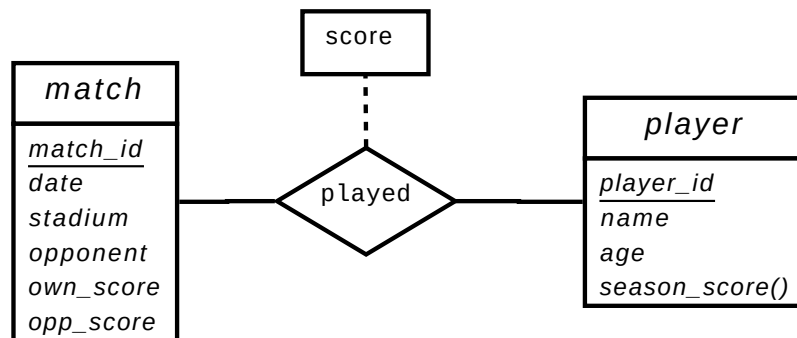


Figure 7.4 E-R diagram for favourite team statistics.

7.3 **Answer:** The diagram is shown in Figure 7.4.

7.4 **Answer:** The different occurrences of an entity may have different sets of attributes, leading to an inconsistent diagram. Instead, the attributes of an entity should be specified only once. All other occurrences of the entity should omit attributes. Since it is not possible to have an entity without any attributes, an occurrence of an entity without attributes clearly indicates that the attributes are specified elsewhere.

7.5 **Answer:**

- a. If a pair of entity sets are connected by a path in an E-R diagram, the entity sets are related, though perhaps indirectly. A disconnected graph implies that there are pairs of entity sets that are unrelated to each other. In an enterprise, we can say that the two departments are completely independent of each other. If we split the graph into connected components, we have, in effect, a separate database corresponding to each connected component.
- b. As indicated in the answer to the previous part, a path in the graph between a pair of entity sets indicates a (possibly indirect) relationship between the two entity sets. If there is a cycle in the graph then every pair of entity sets on the cycle are related to each other in at least two distinct ways. If the E-R diagram is acyclic then there is a unique path between every pair of entity sets and, thus, a unique relationship between every pair of entity sets.

7.6 **Answer:**

- a. Let $E = \{e_1, e_2\}$, $A = \{a_1, a_2\}$, $B = \{b_1\}$, $C = \{c_1\}$, $R_A = \{(e_1, a_1), (e_2, a_2)\}$, $R_B = \{(e_1, b_1)\}$, and $R_C = \{(e_1, c_1)\}$. We see that because of the tuple (e_2, a_2) , no instance of R exists which corresponds to E , R_A , R_B and R_C .

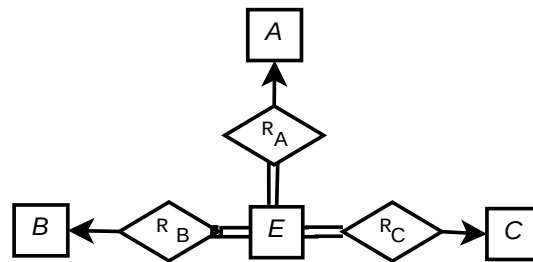


Figure 7.5 E-R diagram for Exercise 7.6b.

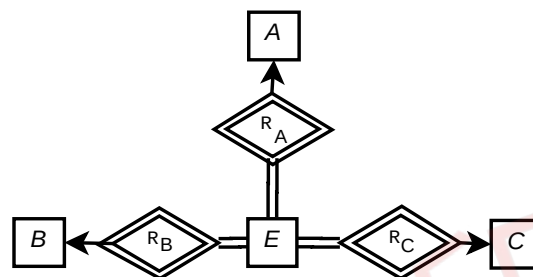


Figure 7.6 E-R diagram for Exercise 7.6d.

- b. See Figure 7.5. The idea is to introduce total participation constraints between E and the relationships R_A , R_B , R_C so that every tuple in E has a relationship with A , B and C .
- c. Suppose A totally participates in the relationship R , then introduce a total participation constraint between A and R_A .
- d. Consider E as a weak entity set and R_A , R_B and R_C as its identifying relationship sets. See Figure 7.6.

7.7 **Answer:** The primary key of a weak entity set can be inferred from its relationship with the strong entity set. If we add primary key attributes to the weak entity set, they will be present in both the entity set and the relationship set and they have to be the same. Hence there will be redundancy.

7.8 **Answer:** In this example, the primary key of *section* consists of the attributes (*course_id*, *semester*, *year*), which would also be the primary key of *sec_course*, while *course_id* is a foreign key from *sec_course* referencing *course*. These constraints ensure that a particular *section* can only correspond to one *course*, and thus the many-to-one cardinality constraint is enforced. However, these constraints cannot enforce a total participation constraint, since a *course* or a *section* may not participate in the *sec_course* relationship.

7.9 **Answer:**

In addition to declaring *s_ID* as primary key for *advisor*, we declare *i_ID* as a super key for *advisor* (this can be done in SQL using the **unique** constraint on *i_ID*).

7.10 Answer: The foreign key attribute in *R* corresponding to primary key of *B* should be made **not null**. This ensures that no tuple of *A* which is not related to any entry in *B* under *R* can come in *R*. For example, say *a* is a tuple in *A* which has no corresponding entry in *R*. This means when *R* is combined with *A*, it would have foreign key attribute corresponding to *B* as **null** which is not allowed.

7.11 Answer:

- a. For the many-to-many case, the relationship must be represented as a separate relation which cannot be combined with either participating entity. Now, there is no way in SQL to ensure that a primary key value occurring in an entity *E1* also occurs in a many-to-many relationship *R*, since the corresponding attribute in *R* is not unique; SQL foreign keys can only refer to the primary key or some other unique key. Similarly, for the one-to-many case, there is no way to ensure that an attribute on the one side appears in the relation corresponding to the many side, for the same reason.
- b. Let the relation *R* be many-to-one from entity *A* to entity *B* with *a* and *b* as their respective primary keys, respectively. We can put the following check constraints on the "one" side relation *B*:

```
constraint total_part check (b in (select b from A));  
set constraints total_part deferred;
```

Note that the constraint should be set to deferred so that it is only checked at the end of the transaction; otherwise if we insert a *b* value in *B* before it is inserted in *A* the above constraint would be violated, and if we insert it in *A* before we insert it in *B*, a foreign key violation would occur.

7.12 Answer: *A* inherits all the attributes of *X* plus it may define its own attributes. Similarly *C* inherits all the attributes of *Y* plus its own attributes. *B* inherits the attributes of both *X* and *Y*. If there is some attribute *name* which belongs to both *X* and *Y*, it may be referred to in *B* by the qualified name *X.name* or *Y.name*.

7.13 Answer:

- a. The E-R diagram is shown in Figure 7.7. The primary key attributes *student_id* and *instructor_id* are assumed to be immutable, that is they are not allowed to change with time. All other attributes are assumed to potentially change with time. Note that the diagram uses multivalued composite attributes such as *valid_times* or *name*, with sub attributes such as *start_time* or *value*.

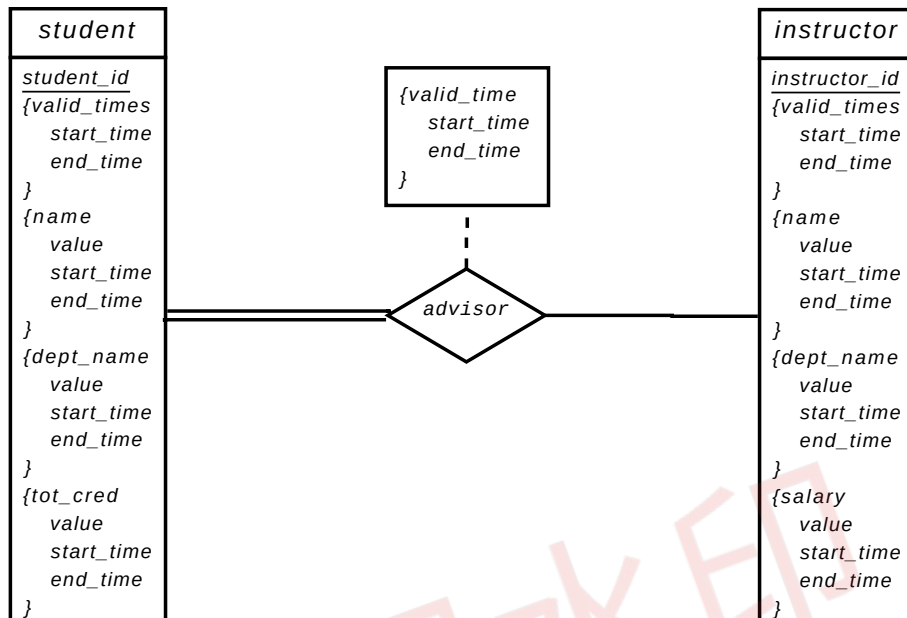


Figure 7.7 E-R diagram for Exercise 7.13

The *value* attribute is a subattribute of several attributes such as *name*, *tot_cred* and *salary*, and refers to the name, total credits or salary during a particular interval of time.

- b. The generated relations are as shown below. Each multivalued attribute has turned into a relation, with the relation name consisting of the original relation name concatenated with the name of the multivalued attribute. The relation corresponding to the entity has only the primary key attribute.

student(*student_id*)
student_valid_times(*student_id*, *start_time*, *end_time*)
student_name(*student_id*, *value*, *start_time*, *end_time*)
student_dept_name(*student_id*, *value*, *start_time*, *end_time*)
student_tot_cred(*student_id*, *value*, *start_time*, *end_time*)
instructor(*instructor_id*)
instructor_valid_times(*instructor_id*, *start_time*, *end_time*)
instructor_name(*instructor_id*, *value*, *start_time*, *end_time*)
instructor_dept_name(*instructor_id*, *value*, *start_time*, *end_time*)
instructor_salary(*instructor_id*, *value*, *start_time*, *end_time*)
advisor(*student_id*, *instructor_id*, *start_time*, *end_time*)

The primary keys shown are derived directly from the E-R diagram. If we add the additional constraint that time intervals cannot overlap (or even the weaker condition that one start time cannot have two end times), we can remove the *end_time* from all the above primary keys.

试用水印

试用水印

CHAPTER 7



Database Design and the E-R Model

This chapter introduces the entity-relationship model in detail. A significant change in the 6th edition is the change in the E-R notation used in the book. There are several alternative E-R notations used in the industry. Increasingly, however, the UML class diagram notation is used instead of the traditional E-R notation used in earlier editions of the book. Among the reasons is the wide availability of UML tools, as well as the conciseness of the UML notation compared to the notation with separate ovals to represent attributes. In keeping with this trend, we have changed our E-R notation to be more compatible with UML.

The chapter covers numerous features of the model, several of which can be omitted depending on the planned coverage of the course. Extended E-R features (Section 7.8) and all subsequent sections may be omitted in case of time constraints in the course, without compromising the students understanding of basic E-R modeling. However, we recommend covering specialization (Section 7.8.1) at least in some detail, since it is widely used in object-oriented modeling.

The E-R model itself and E-R diagrams are used often in the text. It is important that students become comfortable with them. The E-R model is an excellent context for the introduction of students to the complexity of database design. For a given enterprise there are often a wide variety of E-R designs. Although some choices are arbitrary, it is often the case that one design is inherently superior to another. Several of the exercises illustrate this point. The evaluation of the goodness of an E-R design requires an understanding of the enterprise being modeled and the applications to be run. It is often possible to lead students into a debate of the relative merits of competing designs and thus illustrate by example that understanding the application is often the hardest part of database design.

Among the points that are worth discussing when coming up with an E-R design are:

1. Naming of attributes: this is a key aspect of a good design. One approach to design ensures that no two attributes share a name by accident; thus, if ID appears as an attribute of person, it should not appear as an attribute of

another relation, unless it references the ID of person. When natural joins are used in queries, this approach avoids accidental equation of attributes to some extent, although not always; for example, students and instructors share attributes ID and name (presumably inherited from a generalization *person*), so a query that joins the student and instructor relations would equate the respective attribute names.

2. Primary keys: one approach to design creates identifier values for every entity, which are internal to the system and not normally made visible to end users. These internal values are often declared in SQL as **auto increment**, meaning that whenever a tuple is inserted to the relation, a unique value is given to the attribute automatically.

In contrast, the alternative approach, which we have used in this book, avoids creating artificial internal identifiers, and instead uses externally visible attributes as primary key values wherever possible.

As an example, in any university employees and students have externally visible identifiers. These could be used as the primary keys, or alternatively, the application can create identifiers that are not externally visible, and use them as the value for the primary key.

As another example, the *section* table, which has the combination of (*course_id*, *section_id*, *semester*, *year*) as primary key, could instead have a section identifier that is unique across all sections as primary key, with the *course_id*, *section_id*, *semester*, *year* as non-primary key attributes. The difference would be that the relations that refer to *section*, namely *teaches* and *takes*, would have a single unique section id attribute as a foreign key referring to *section*, and would not need to store *course_id*, *section_id*, *semester*, and *year*.

Considerable emphasis is placed on the construction of tables from E-R diagrams. This serves to build intuition for the discussion of the relational model in the subsequent chapters. It also serves to ground abstract concepts of entities and relationships into the more concrete concepts of relations. Several other texts place this material along with the relational data model, rather than in the E-R model chapter. Our motivation for placing this material here is help students to appreciate how E-R data models get used in reality, while studying the E-R model rather than later on.

Exercises

- 7.14 Explain the distinctions among the terms primary key, candidate key, and superkey.

Answer: A *superkey* is a set of one or more attributes that, taken collectively, allows us to identify uniquely an entity in the entity set. A superkey may contain extraneous attributes. If *K* is a superkey, then so is any superset of *K*. A superkey for which no proper subset is also a superkey is called a *candidate key*. It is possible that several distinct sets of attributes could

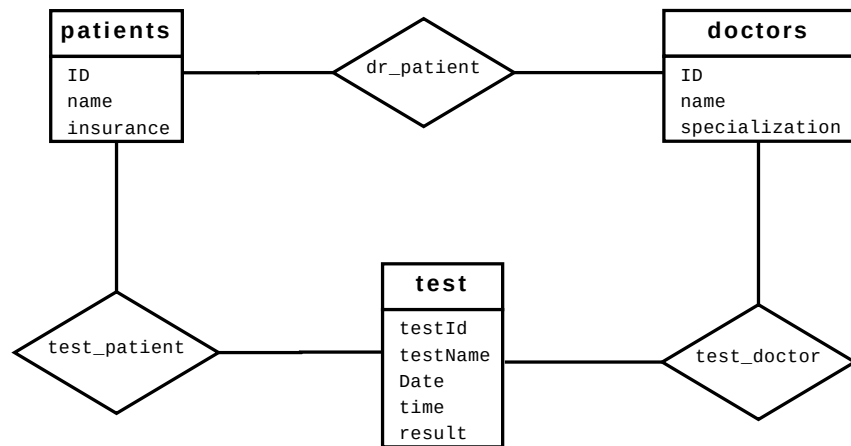


Figure 7.1 E-R diagram for a hospital.

serve as candidate keys. The *primary key* is one of the candidate keys that is chosen by the database designer as the principal means of identifying entities within an entity set.

- 7.15 Construct an E-R diagram for a hospital with a set of patients and a set of medical doctors. Associate with each patient a log of the various tests and examinations conducted.

Answer:

An E-R diagram for the hospital is shown in Figure 7.1. Although the diagram meets the specifications of the question, a real-world hospital would have many more requirements, such as tracking patient admissions and visits, including which doctor sees a patient on each visit, recording results of tests in a more structured manner, and so on.

- 7.16 Construct appropriate relation schemas for each of the E-R diagrams in Practice Exercises 7.1 to 7.3.

Answer:

- a. Car insurance tables:

customer (*customer_id*, *name*, *address*)

car (*license*, *model*)

owns(*customer_id*, *license_no*)

accident (*report_id*, *date*, *place*)

participated(*license_no*, *report_id*) *policy*(*policy_id*)

covers(*policy_id*, *license_no*)

premium_payment(*policy_id*, *payment_no*, *due_date*, *amount*, *received_on*)

Note that a more realistic database design would include details of who was driving the car when an accident happened, and the damage amount for each car that participated in the accident.

- b. Student Exam tables:
 i. Ternary Relationship:

student (*student_id*, *name*, *dept_name*, *tot_cred*)
course (*course_id*, *title*, *credits*)
section (*course_id*, *section_id*, *semester*, *year*)
exam (*exam_id*, *name*, *place*, *time*)
exam_marks (*student_id*, *course_id*, *section_id*, *semester*, *year*, *exam_id*, *marks*)

- ii. Binary relationship:

student (*ID*, *name*, *dept_name*, *tot_cred*)
course (*course_id*, *title*, *credits*)
section (*course_id*, *section_id*, *semester*, *year*)
exam_marks (*student_id*, *course_id*, *sec_id*, *semester*, *year*, *exam_id*, *marks*)

- c. Player Match tables:

match (*match_id*, *date*, *stadium*, *opponent*, *own_score*, *opp_score*)
player (*player_id*, *name*, *age*, *season_score*)
played (*match_id*, *player_id*, *score*)

- 7.17 Extend the E-R diagram of Practice Exercise 7.3 to track the same information for all teams in a league.

Answer: See Figure 7.2. Note that we assume a player can play in only one team; if a player may switch teams, we would have to track for each match which team the player was in, which we could do by turning the relationship *played* into a ternary relationship.

- 7.18 Explain the difference between a weak and a strong entity set.

Answer: A strong entity set has a primary key. All tuples in the set are distinguishable by that key. A weak entity set has no primary key unless attributes of the strong entity set on which it depends are included. Tuples in a weak entity set are partitioned according to their relationship with tuples in a strong entity set. Tuples within each partition are distinguishable by a discriminator, which is a set of attributes.

- 7.19 We can convert any weak entity set to a strong entity set by simply adding appropriate attributes. Why, then, do we have weak entity sets?

Answer: We have weak entities for several reasons:

- We want to avoid the data duplication and consequent possible inconsistencies caused by duplicating the key of the strong entity.
- Weak entities reflect the logical structure of an entity being dependent on another entity.
- Weak entities can be deleted automatically when their strong entity is deleted.

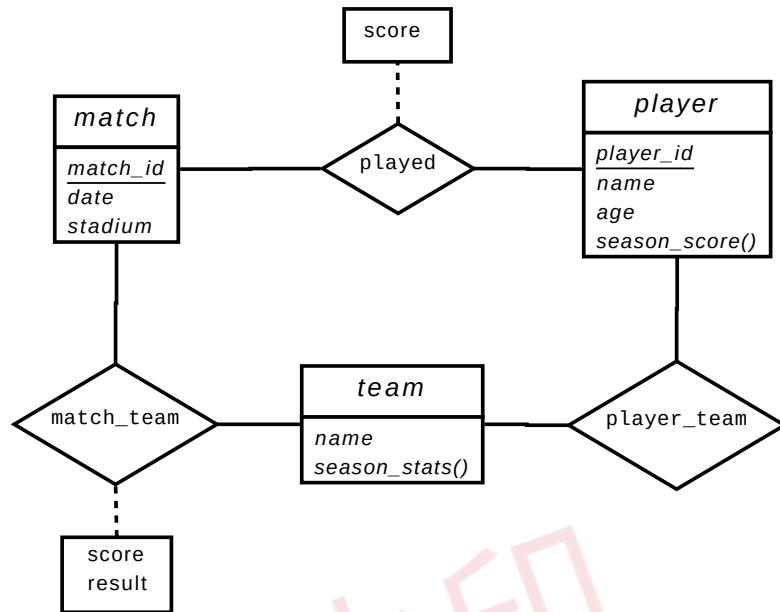


Figure 7.2 E-R diagram for all teams statistics.

- Weak entities can be stored physically with their strong entities.

7.20 Consider the E-R diagram in Figure 7.29, which models an online bookstore.

- List the entity sets and their primary keys.
- Suppose the bookstore adds Blu-ray discs and downloadable video to its collection. The same item may be present in one or both formats, with differing prices. Extend the E-R diagram to model this addition, ignoring the effect on shopping baskets.
- Now extend the E-R diagram, using generalization, to model the case where a shopping basket may contain any combination of books, Blu-ray discs, or downloadable video.

Answer: Interpret the second part of the question as the bookstore adds videos, which may be in Blu-ray disk format or in downloadable format; the same video may be present in both formats.

- Entity sets:

author(name, address, URL)
publisher(name, address, phone, URL)
book(ISBN, title, year, price)
customer(email, name, address, phone)
shopping_basket(basket_id)
warehouse(code, address, phone)

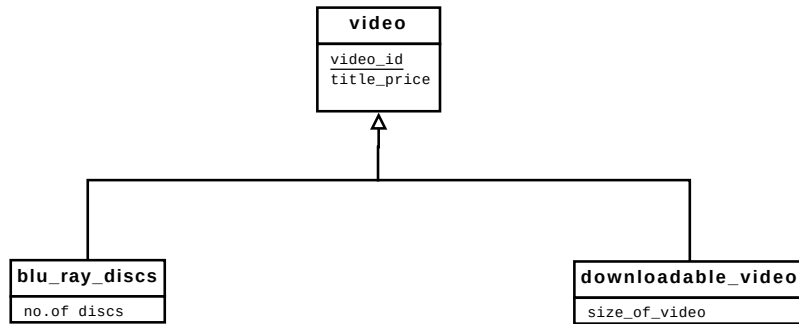


Figure 7.3 ER Diagram for Exercise 7.20 (b)

- b. The ER-diagram portion related to videos is shown in Figure 7.3.
- c. The E-R diagram shown in Figure 7.4 should be added to the E-R diagram of Figure 7.29. Entities that are shown already in Figure 7.29 are shown with only their names, omitting the attributes. The *contains* relationship in Figure 7.29 should be replaced by the version in Figure 7.4. All other parts of Figure 7.29 remain unchanged.

7.21 Design a database for an automobile company to provide to its dealers to assist them in maintaining customer records and dealer inventory and to assist sales staff in ordering cars.

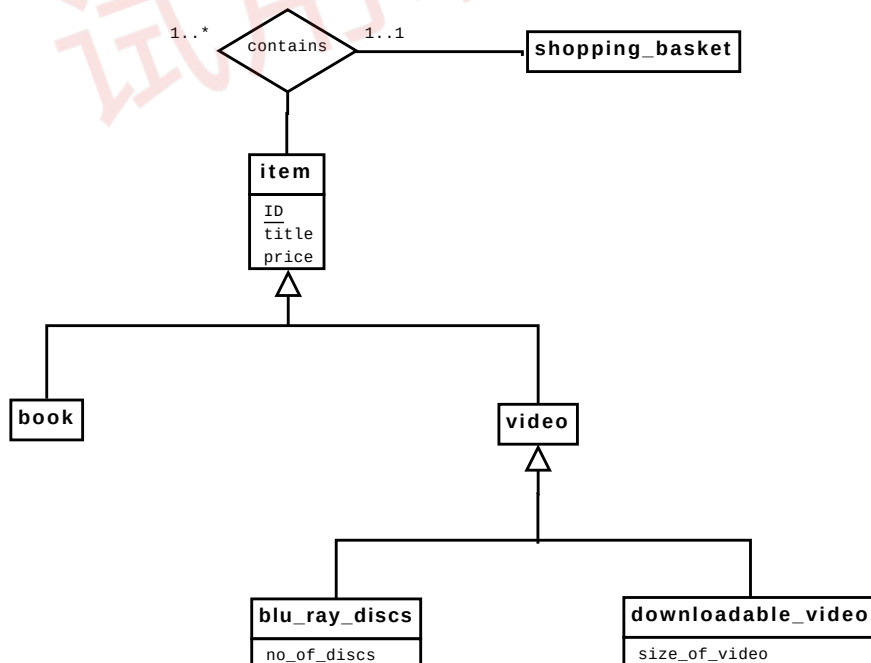


Figure 7.4 ER Diagram for Exercise 7.20 (c)

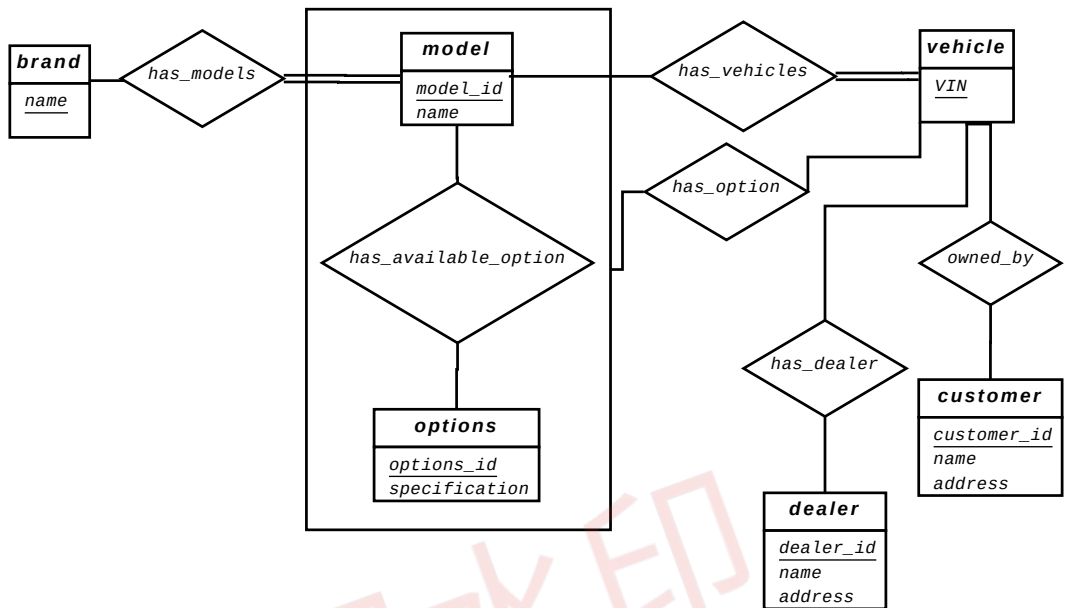


Figure 7.5 ER Diagram for Exercise 7.21

Each vehicle is identified by a vehicle identification number (VIN). Each individual vehicle is a particular model of a particular brand offered by the company (e.g., the XF is a model of the car brand Jaguar of Tata Motors). Each model can be offered with a variety of options, but an individual car may have only some (or none) of the available options. The database needs to store information about models, brands, and options, as well as information about individual dealers, customers, and cars.

Your design should include an E-R diagram, a set of relational schemas, and a list of constraints, including primary-key and foreign-key constraints.

Answer:

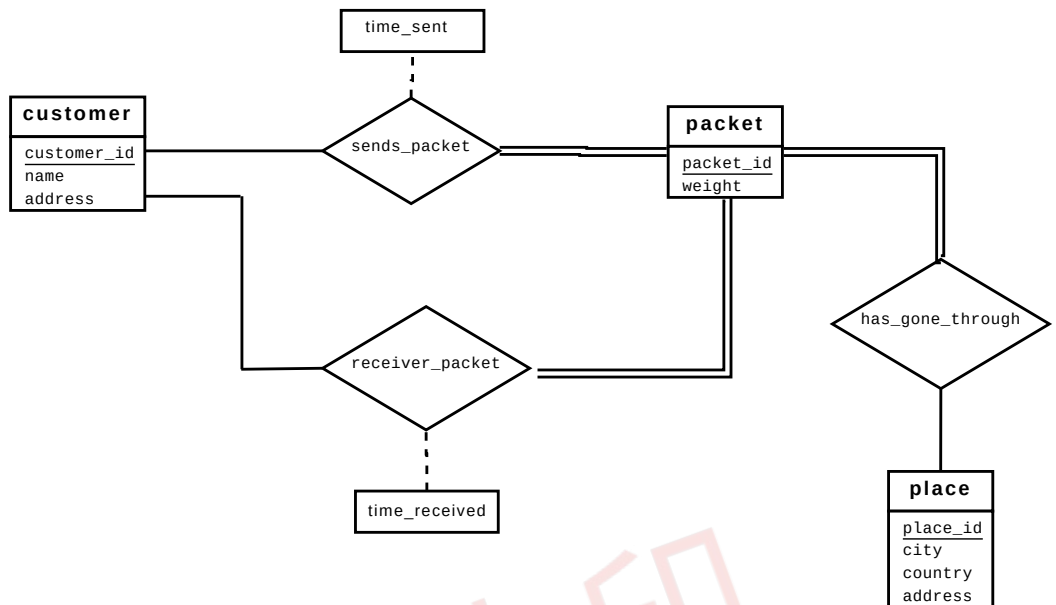
The E-R diagram is shown in Figure 7.5. Note that the *has_option* relationship links a vehicle to an aggregation on the relationship *has_available_option*, instead of directly to the entity set *options*, to ensure that a particular vehicle instance cannot get an option that does not correspond to its model. The alternative of directly linking to *options* is acceptable if ensuring the above integrity constraint is not critical.

The relational schema, along with primary-key and foreign-key constraints is shown below.

```

brand(name)
model(model_id,
      name)
vehicle(VIN)
option(option_id,
       specification)
customer(customer_id,
        name,
        address)
dealer(dealer_id,
       name,
       address)
has_models(name,
           model_id,
           foreign key name references brand ,
           foreign key model_id references model
)
has_vehicles(model_id,
            VIN,
            foreign key VIN references vehicle,
            foreign key model_id references model
)
available_options(model_id,
                 option_id,
                 foreign key option_id references option,
                 foreign key model_id references model
)
has_options(VIN,
           model_id,
           option_id,
           foreign key VIN references vehicle,
           foreign key (model_id, option_id) references available_options
)
has_dealer(VIN,
          dealer_id,
          foreign key dealer_id references dealer,
          foreign key VIN references vehicle
)
owned_by(VIN,
        customer_id,
        foreign key customer_id references customer,
        foreign key VIN references vehicle
)

```

As an alternative:

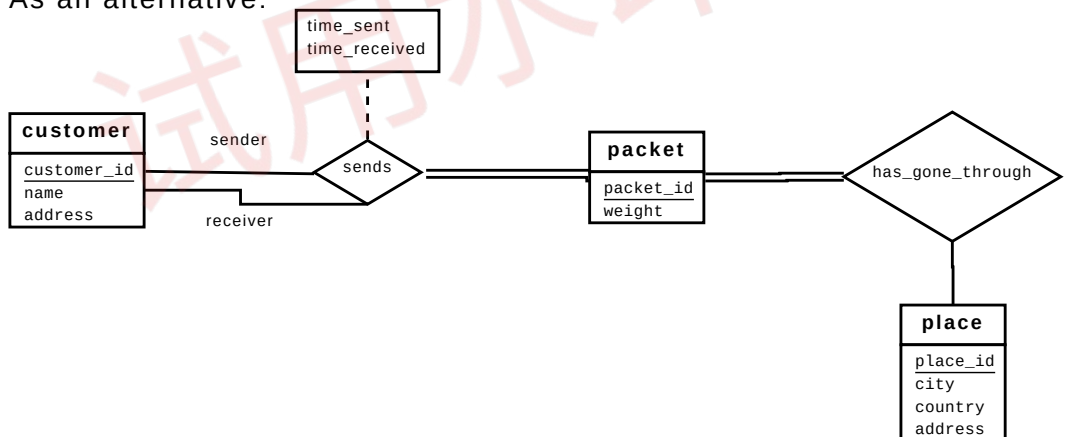


Figure 7.6 ER Diagram Alternatives for Exercise 7.22

- 7.22** Design a database for a world-wide package delivery company (e.g., DHL or FedEx). The database must be able to keep track of customers (who ship items) and customers (who receive items); some customers may do both. Each package must be identifiable and trackable, so the database must be able to store the location of the package and its history of locations. Locations include trucks, planes, airports, and warehouses.

Your design should include an E-R diagram, a set of relational schemas, and a list of constraints, including primary-key and foreign-key constraints.

Answer:

Two alternative E-R diagrams are shown in Figure 7.6. The relational schema, including primary-key and foreign-key constraints, corresponding to the second alternative is shown below.

```
customer(customer_id,
        name,
        address)
packet(packet_id,
       weight)
place(place_id,
      city,
      country,
      address)
sends(sender_id,
      receiver_id,
      packet_id,
      time_received,
      time_sent
      foreign key sender_id references customer,
      foreign key receiver_id references customer,
      foreign key packet_id references packet
)
has_gone_through(
  packet_id,
  place_id
  foreign key packet_id references packet,
  foreign key place_id references place
)
```

- 7.23** Design a database for an airline. The database must keep track of customers and their reservations, flights and their status, seat assignments on individual flights, and the schedule and routing of future flights.

Your design should include an E-R diagram, a set of relational schemas, and a list of constraints, including primary-key and foreign-key constraints.

Answer:

The E-R diagram is shown in Figure 7.7. We assume that the schedule of a flight is fixed across time, although we allow specification of on which days a flight is scheduled. For a particular instance of a flight however we record actual times of departure and arrival. In reality, schedules change with time, so the schedule and routing should be for a particular flight for specified dates, or for a specified range of dates; we ignore this complexity.

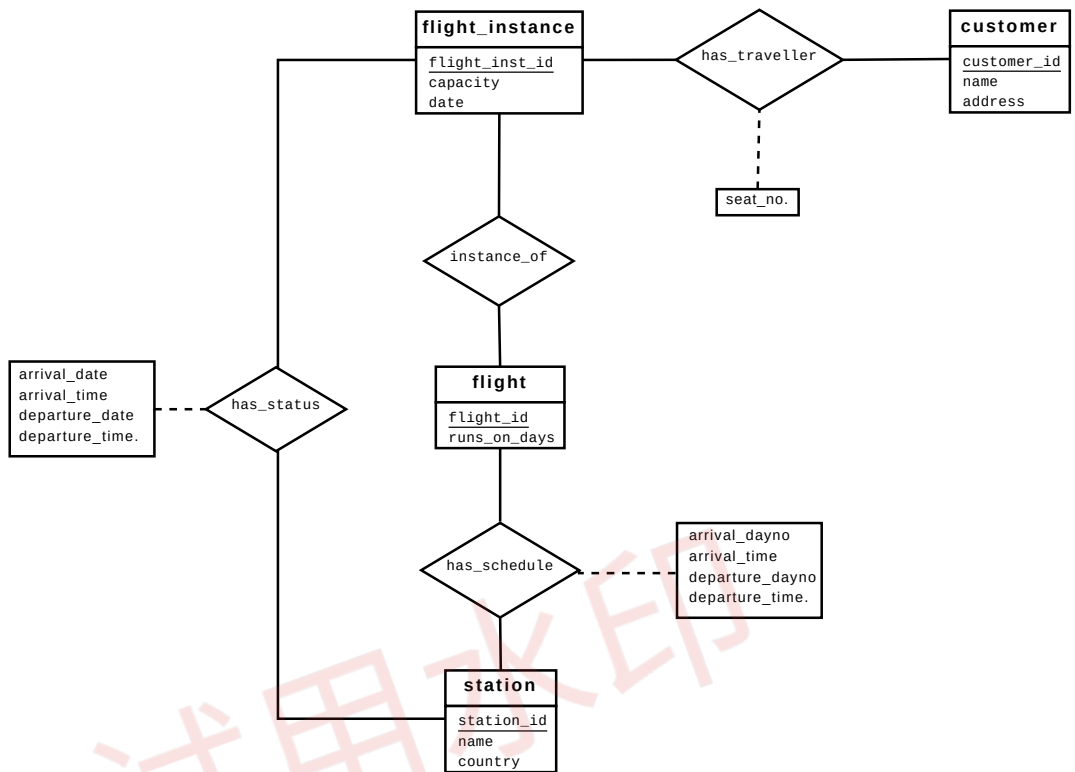


Figure 7.7 ER Diagram for Exercise 7.23

```

flight_instance(flight_inst_id, capacity, date)
customer(customer_id, name, address)
flight(flight_id, runs_on_days)
station(station_id, name, country)
has_traveller(
    flight_inst_id,
    customer_id,
    seat_number,
    foreign key flight_inst_id references flight_instance,
    foreign key customer_id references customer
)
instance_of(
    flight_inst_id,
    flight_id,
    foreign key flight_inst_id references flight_instance,
    foreign key flight_id references flight
)

```

```

has_schedule(
    flight_id,
    station_id,
    order,
    arrival_dayno,
    arrival_time,
    departure_dayno,
    departure_time,
    foreign key flight_id references flight,
    foreign key station_id references station
)
has_status(
    flight_inst_id,
    station_id,
    arrival_date,
    arrival_time,
    departure_date,
    departure_time,
    foreign key flight_inst_id references flight_instance,
    foreign key station_id references station
)

```

- 7.24 In Section 7.7.3, we represented a ternary relationship (repeated in Figure 7.27a) using binary relationships, as shown in Figure 7.27b. Consider the alternative shown in Figure 7.27c. Discuss the relative merits of these two alternative representations of a ternary relationship by binary relationships.

Answer: In the model of Figure 7.27b, there can be instances where E , A , B , C , R_A , R_B and R_C cannot correspond to any instance of A , B , C and R .

The model of Figure 7.27c will not be able to represent all ternary relationships. Consider the ABC relationship set below.

A	B	C
1	2	3
4	2	7
4	8	3

If ABC is broken into three relationships sets AB , BC and AC , the three will imply that the relation $(4, 2, 3)$ is a part of ABC .

- 7.25 Consider the relation schemas shown in Section 7.6, which were generated from the E-R diagram in Figure 7.15. For each schema, specify what foreign-key constraints, if any, should be created.

Answer: The foreign-key constraints are as specified below.

```

teaches(
    foreign key ID references instructor,
    foreign key (course_id, sec_id, semester, year) references sec_course
)

takes(
    foreign key ID references student,
    foreign key (course_id, sec_id, semester, year) references sec_course
)

prereq(
    foreign key course_id references course,
    foreign key prereq_id references course
)

advisor(
    foreign key s_ID references student,
    foreign key i_id references instructor
)

sec_course(
    foreign key course_id references course,
    foreign key (sec_id, semester, year) references section
)

sec_time_slot(
    foreign key (course_id, sec_id, semester, year) references sec_course
    foreign key time_slot_id references time_slot
)

sec_class(
    foreign key (course_id, sec_id, semester, year) references sec_course
    foreign key (building, room_number) references classroom
)

inst_dept(
    foreign key ID references instructor
    foreign key dept_name references department
)

stud_dept(
    foreign key ID references student
    foreign key dept_name references department
)

```

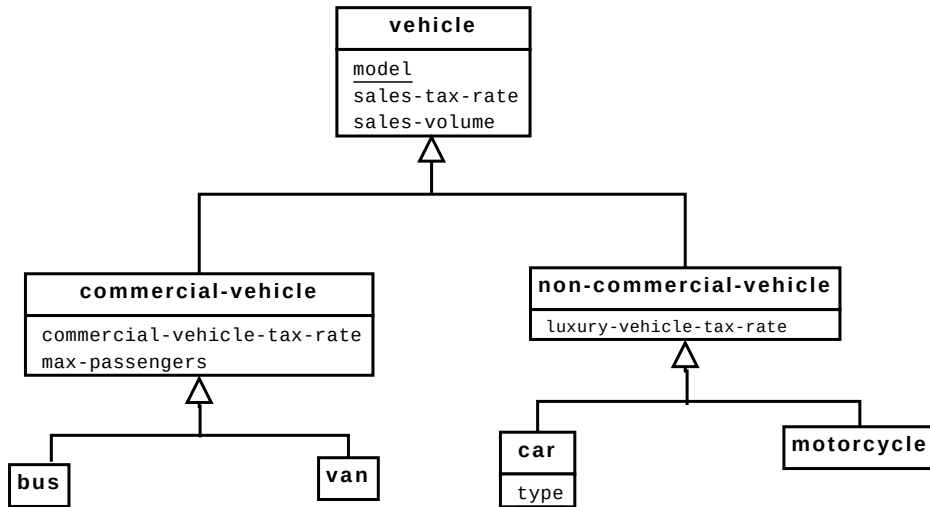


Figure 7.8 E-R diagram of motor-vehicle sales company.

```

course_dept(
    foreign key course_id references course
    foreign key dept_name references department
)

```

- 7.26 Design a generalization–specialization hierarchy for a motor vehicle sales company. The company sells motorcycles, passenger cars, vans, and buses. Justify your placement of attributes at each level of the hierarchy. Explain why they should not be placed at a higher or lower level.

Answer: Figure 7.8 gives one possible hierarchy; note that there could be many alternative solutions. The generalization–specialization hierarchy for the motor-vehicle company is given in the figure. *model*, *sales-tax-rate* and *sales-volume* are attributes necessary for all types of vehicles. Commercial vehicles attract commercial vehicle tax, and each kind of commercial vehicle has a passenger carrying capacity specified for it. Some kinds of non-commercial vehicles attract luxury vehicle tax. Cars alone can be of several types, such as sports-car, sedan, wagon etc., hence the attribute *type*.

- 7.27 Explain the distinction between condition-defined and user-defined constraints. Which of these constraints can the system check automatically? Explain your answer.

Answer: In a generalization–specialization hierarchy, it must be possible to decide which entities are members of which lower level entity sets. In a condition-defined design constraint, membership in the lower level entity-sets is evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate. User-defined lower-level entity sets are not constrained by a membership condition; rather, entities are assigned to a given entity set by the database user.

Condition-defined constraints alone can be automatically handled by the system. Whenever any tuple is inserted into the database, its membership in the various lower level entity-sets can be automatically decided by evaluating the respective membership predicates. Similarly when a tuple is updated, its membership in the various entity sets can be re-evaluated automatically.

- 7.28 Explain the distinction between disjoint and overlapping constraints.

Answer: In a disjointness design constraint, an entity can belong to not more than one lower-level entity set. In overlapping generalizations, the same entity may belong to more than one lower-level entity sets. For example, in the employee-workteam example of the book, a manager may participate in more than one work-team.

- 7.29 Explain the distinction between total and partial constraints.

Answer: In a generalization–specialization hierarchy, a total constraint means that an entity belonging to the higher level entity set must belong to the lower level entity set. A partial constraint means that an entity belonging to the higher level entity set may or may not belong to the lower level entity set.

试用水印

CHAPTER 8



Relational Database Design

Exercises

- 8.1 Suppose that we decompose the schema $R = (A, B, C, D, E)$ into

(A, B, C)
 $(A, D, E).$

Show that this decomposition is a lossless-join decomposition if the following set F of functional dependencies holds:

$A \rightarrow BC$
 $CD \rightarrow E$
 $B \rightarrow D$
 $E \rightarrow A$

Answer: A decomposition $\{R_1, R_2\}$ is a lossless-join decomposition if $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$. Let $R_1 = (A, B, C)$, $R_2 = (A, D, E)$, and $R_1 \cap R_2 = A$. Since A is a candidate key (see Practice Exercise 8.6), Therefore $R_1 \cap R_2 \rightarrow R_1$.

- 8.2 List all functional dependencies satisfied by the relation of Figure 8.17.

Answer: The nontrivial functional dependencies are: $A \rightarrow B$ and $C \rightarrow B$, and a dependency they logically imply: $AC \rightarrow B$. There are 19 trivial functional dependencies of the form $\alpha \rightarrow \beta$, where $\beta \subseteq \alpha$. C does not functionally determine A because the first and third tuples have the same C but different A values. The same tuples also show B does not functionally determine A . Likewise, A does not functionally determine C because the first two tuples have the same A value and different C values. The same tuples also show B does not functionally determine C .

- 8.3 Explain how functional dependencies can be used to indicate the following:

- A one-to-one relationship set exists between entity sets *student* and *instructor*.
- A many-to-one relationship set exists between entity sets *student* and *instructor*.

Answer: Let $Pk(r)$ denote the primary key attribute of relation r .

- The functional dependencies $Pk(student) \rightarrow Pk(instructor)$ and $Pk(instructor) \rightarrow Pk(student)$ indicate a one-to-one relationship because any two tuples with the same value for student must have the same value for instructor, and any two tuples agreeing on instructor must have the same value for student.
- The functional dependency $Pk(student) \rightarrow Pk(instructor)$ indicates a many-to-one relationship since any student value which is repeated will have the same instructor value, but many student values may have the same instructor value.

- 8.4 Use Armstrong's axioms to prove the soundness of the union rule. (Hint: Use the augmentation rule to show that, if $\alpha \rightarrow \beta$, then $\alpha \rightarrow \alpha\beta$. Apply the augmentation rule again, using $\alpha \rightarrow \gamma$, and then apply the transitivity rule.)

Answer: To prove that:

$$\text{if } \alpha \rightarrow \beta \text{ and } \alpha \rightarrow \gamma \text{ then } \alpha \rightarrow \beta\gamma$$

Following the hint, we derive:

$\alpha \rightarrow \beta$	given
$\alpha\alpha \rightarrow \alpha\beta$	augmentation rule
$\alpha \rightarrow \alpha\beta$	union of identical sets
$\alpha \rightarrow \gamma$	given
$\alpha\beta \rightarrow \gamma\beta$	augmentation rule
$\alpha \rightarrow \beta\gamma$	transitivity rule and set union commutativity

- 8.5 Use Armstrong's axioms to prove the soundness of the pseudotransitivity rule.

Answer: Proof using Armstrong's axioms of the Pseudotransitivity Rule:
if $\alpha \rightarrow \beta$ and $\gamma\beta \rightarrow \delta$, then $\alpha\gamma \rightarrow \delta$.

$\alpha \rightarrow \beta$	given
$\alpha\gamma \rightarrow \gamma\beta$	augmentation rule and set union commutativity
$\gamma\beta \rightarrow \delta$	given
$\alpha\gamma \rightarrow \delta$	transitivity rule

- 8.6 Compute the closure of the following set F of functional dependencies for relation schema $R = (A, B, C, D, E)$.

$$\begin{aligned}
 A &\rightarrow BC \\
 CD &\rightarrow E \\
 B &\rightarrow D \\
 E &\rightarrow A
 \end{aligned}$$

List the candidate keys for R .

Answer: Note: It is not reasonable to expect students to enumerate all of F^+ . Some shorthand representation of the result should be acceptable as long as the nontrivial members of F^+ are found.

Starting with $A \rightarrow BC$, we can conclude: $A \rightarrow B$ and $A \rightarrow C$.

Since $A \rightarrow B$ and $B \rightarrow D$, $A \rightarrow D$ (decomposition, transitive)

Since $A \rightarrow CD$ and $CD \rightarrow E$, $A \rightarrow E$ (union, decomposition, transitive)

Since $A \rightarrow A$, we have (reflexive)

$A \rightarrow ABCDE$ from the above steps (union)

Since $E \rightarrow A$, $E \rightarrow ABCDE$ (transitive)

Since $CD \rightarrow E$, $CD \rightarrow ABCDE$ (transitive)

Since $B \rightarrow D$ and $BC \rightarrow CD$, $BC \rightarrow ABCDE$ (augmentative, transitive)

Also, $C \rightarrow C$, $D \rightarrow D$, $BD \rightarrow D$, etc.

Therefore, any functional dependency with A , E , BC , or CD on the left hand side of the arrow is in F^+ , no matter which other attributes appear in the FD. Allow $*$ to represent any set of attributes in R , then F^+ is $BD \rightarrow B$, $BD \rightarrow D$, $C \rightarrow C$, $D \rightarrow D$, $BD \rightarrow BD$, $B \rightarrow D$, $B \rightarrow B$, $B \rightarrow BD$, and all FDs of the form $A* \rightarrow \alpha$, $BC* \rightarrow \alpha$, $CD* \rightarrow \alpha$, $E* \rightarrow \alpha$ where α is any subset of $\{A, B, C, D, E\}$. The candidate keys are A , BC , CD , and E .

- 8.7 Using the functional dependencies of Practice Exercise 8.6, compute the canonical cover F_c .

Answer: The given set of FDs F is:-

$$\begin{aligned}
 A &\rightarrow BC \\
 CD &\rightarrow E \\
 B &\rightarrow D \\
 E &\rightarrow A
 \end{aligned}$$

The left side of each FD in F is unique. Also none of the attributes in the left side or right side of any of the FDs is extraneous. Therefore the canonical cover F_c is equal to F .

- 8.8 Consider the algorithm in Figure 8.18 to compute α^+ . Show that this algorithm is more efficient than the one presented in Figure 8.8 (Section 8.4.2) and that it computes α^+ correctly.

Answer: The algorithm is correct because:

- If A is added to *result* then there is a proof that $\alpha \rightarrow A$. To see this, observe that $\alpha \rightarrow \alpha$ trivially so α is correctly part of *result*. If $A \notin \alpha$ is added to *result* there must be some FD $\beta \rightarrow \gamma$ such that $A \in \gamma$ and β is already a subset of *result*. (Otherwise *fdcount* would be nonzero and the **if** condition would be false.) A full proof can be given by induction on the depth of recursion for an execution of **addin**, but such a proof can be expected only from students with a good mathematical background.
- If $A \in \alpha^+$, then A is eventually added to *result*. We prove this by induction on the length of the proof of $\alpha \rightarrow A$ using Armstrong's axioms. First observe that if procedure **addin** is called with some argument β , all the attributes in β will be added to *result*. Also if a particular FD's *fdcount* becomes 0, all the attributes in its tail will definitely be added to *result*. The base case of the proof, $A \in \alpha \Rightarrow A \in \alpha^+$, is obviously true because the first call to **addin** has the argument α . The inductive hypotheses is that if $\alpha \rightarrow A$ can be proved in n steps or less then $A \in \text{result}$. If there is a proof in $n + 1$ steps that $\alpha \rightarrow A$, then the last step was an application of either reflexivity, augmentation or transitivity on a fact $\alpha \rightarrow \beta$ proved in n or fewer steps. If reflexivity or augmentation was used in the $(n + 1)^{\text{st}}$ step, A must have been in *result* by the end of the n^{th} step itself. Otherwise, by the inductive hypothesis $\beta \subseteq \text{result}$. Therefore the dependency used in proving $\beta \rightarrow \gamma$, $A \in \gamma$ will have *fdcount* set to 0 by the end of the n^{th} step. Hence A will be added to *result*.

To see that this algorithm is more efficient than the one presented in the chapter note that we scan each FD once in the main program. The resulting array *appears* has size proportional to the size of the given FDs. The recursive calls to **addin** result in processing linear in the size of *appears*. Hence the algorithm has time complexity which is linear in the size of the given FDs. On the other hand, the algorithm given in the text has quadratic time complexity, as it may perform the loop as many times as the number of FDs, in each loop scanning all of them once.

- 8.9 Given the database schema $R(a, b, c)$, and a relation r on the schema R , write an SQL query to test whether the functional dependency $b \rightarrow c$ holds on relation r . Also write an SQL assertion that enforces the functional dependency. Assume that no null values are present. (Although part of the SQL standard, such assertions are not supported by any database implementation currently.)

Answer:

- a. The query is given below. Its result is non-empty if and only if $b \rightarrow c$ does not hold on r .

```
select b
from r
group by b
having count(distinct c) > 1
```

- b.

```
create assertion b_to_c check
(not exists
  (select b
   from r
   group by b
   having count(distinct c) > 1
  )
)
```

- 8.10 Our discussion of lossless-join decomposition implicitly assumed that attributes on the left-hand side of a functional dependency cannot take on null values. What could go wrong on decomposition, if this property is violated?

Answer: The natural join operator is defined in terms of the cartesian product and the selection operator. The selection operator, gives *unknown* for any query on a null value. Thus, the natural join excludes all tuples with null values on the common attributes from the final result. Thus, the decomposition would be lossy (in a manner different from the usual case of lossy decomposition), if null values occur in the left-hand side of the functional dependency used to decompose the relation. (Null values in attributes that occur only in the right-hand side of the functional dependency do not cause any problems.)

- 8.11 In the BCNF decomposition algorithm, suppose you use a functional dependency $\alpha \rightarrow \beta$ to decompose a relation schema $r(\alpha, \beta, \gamma)$ into $r_1(\alpha, \beta)$ and $r_2(\alpha, \gamma)$.

- What primary and foreign-key constraint do you expect to hold on the decomposed relations?
- Give an example of an inconsistency that can arise due to an erroneous update, if the foreign-key constraint were not enforced on the decomposed relations above.
- When a relation is decomposed into 3NF using the algorithm in Section 8.5.2, what primary and foreign key dependencies would you expect will hold on the decomposed schema?

Answer:

- a. α should be a primary key for r_1 , and α should be the foreign key from r_2 , referencing r_1 .
- b. If the foreign key constraint is not enforced, then a deletion of a tuple from r_1 would not have a corresponding deletion from the referencing tuples in r_2 . Instead of deleting a tuple from r , this would amount to simply setting the value of α to null in some tuples.
- c. For every schema $r_i(\alpha\beta)$ added to the schema because of a rule $\alpha \rightarrow \beta$, α should be made the primary key. Also, a candidate key γ for the original relation is located in some newly created relation r_k , and is a primary key for that relation.
Foreign key constraints are created as follows: for each relation r_i created above, if the primary key attributes of r_i also occur in any other relation r_j , then a foreign key constraint is created from those attributes in r_j , referencing (the primary key of) r_i .

8.12 Let $R_1, R_2, \dots, R_n$ be a decomposition of schema U . Let $u(U)$ be a relation, and let $r_i = \Pi_{R_i}(u)$. Show that

$$u \subseteq r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$$

Answer: Consider some tuple t in u .

Note that $r_i = \Pi_{R_i}(u)$ implies that $t[R_i] \in r_i$, $1 \leq i \leq n$. Thus,

$$t[R_1] \bowtie t[R_2] \bowtie \dots \bowtie t[R_n] \in r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$$

By the definition of natural join,

$$t[R_1] \bowtie t[R_2] \bowtie \dots \bowtie t[R_n] = \Pi_{\alpha}(\sigma_{\beta}(t[R_1] \times t[R_2] \times \dots \times t[R_n]))$$

where the condition β is satisfied if values of attributes with the same name in a tuple are equal and where $\alpha = U$. The cartesian product of single tuples generates one tuple. The selection process is satisfied because all attributes with the same name must have the same value since they are projections from the same tuple. Finally, the projection clause removes duplicate attribute names.

By the definition of decomposition, $U = R_1 \cup R_2 \cup \dots \cup R_n$, which means that all attributes of t are in $t[R_1] \bowtie t[R_2] \bowtie \dots \bowtie t[R_n]$. That is, t is equal to the result of this join.

Since t is any arbitrary tuple in u ,

$$u \subseteq r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$$

8.13 Show that the decomposition in Practice Exercise 8.1 is not a dependency-preserving decomposition.

Answer: The dependency $B \rightarrow D$ is not preserved. F_1 , the restriction of F to (A, B, C) is $A \rightarrow ABC$, $A \rightarrow AB$, $A \rightarrow AC$, $A \rightarrow BC$,

$A \rightarrow B, A \rightarrow C, A \rightarrow A, B \rightarrow B, C \rightarrow C, AB \rightarrow AC, AB \rightarrow ABC, AB \rightarrow BC, AB \rightarrow AB, AB \rightarrow A, AB \rightarrow B, AB \rightarrow C, AC$ (same as AB), BC (same as AB), ABC (same as AB). F_2 , the restriction of F to (C, D, E) is $A \rightarrow ADE, A \rightarrow AD, A \rightarrow AE, A \rightarrow DE, A \rightarrow A, A \rightarrow D, A \rightarrow E, D \rightarrow D, E$ (same as A), AD, AE, DE, ADE (same as A). $(F_1 \cup F_2)^+$ is easily seen not to contain $B \rightarrow D$ since the only FD in $F_1 \cup F_2$ with B as the left side is $B \rightarrow B$, a trivial FD. We shall see in Practice Exercise 8.15 that $B \rightarrow D$ is indeed in F^+ . Thus $B \rightarrow D$ is not preserved. Note that $CD \rightarrow ABCDE$ is also not preserved.

A simpler argument is as follows: F_1 contains no dependencies with D on the right side of the arrow. F_2 contains no dependencies with B on the left side of the arrow. Therefore for $B \rightarrow D$ to be preserved there must be an FD $B \rightarrow \alpha$ in F_1^+ and $\alpha \rightarrow D$ in F_2^+ (so $B \rightarrow D$ would follow by transitivity). Since the intersection of the two schemes is A , $\alpha = A$. Observe that $B \rightarrow A$ is not in F_1^+ since $B^+ = BD$.

- 8.14** Show that it is possible to ensure that a dependency-preserving decomposition into 3NF is a lossless-join decomposition by guaranteeing that at least one schema contains a candidate key for the schema being decomposed. (*Hint*: Show that the join of all the projections onto the schemas of the decomposition cannot have more tuples than the original relation.)

Answer: Let F be a set of functional dependencies that hold on a schema R . Let $\sigma = \{R_1, R_2, \dots, R_n\}$ be a dependency-preserving 3NF decomposition of R . Let X be a candidate key for R .

Consider a legal instance r of R . Let $j = \Pi_X(r) \bowtie \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \dots \bowtie \Pi_{R_n}(r)$. We want to prove that $r = j$.

We claim that if t_1 and t_2 are two tuples in j such that $t_1[X] = t_2[X]$, then $t_1 = t_2$. To prove this claim, we use the following inductive argument – Let $F' = F_1 \cup F_2 \cup \dots \cup F_n$, where each F_i is the restriction of F to the schema R_i in σ . Consider the use of the algorithm given in Figure 8.8 to compute the closure of X under F' . We use induction on the number of times that the *for* loop in this algorithm is executed.

- *Basis* : In the first step of the algorithm, *result* is assigned to X , and hence given that $t_1[X] = t_2[X]$, we know that $t_1[\text{result}] = t_2[\text{result}]$ is true.
- *Induction Step* : Let $t_1[\text{result}] = t_2[\text{result}]$ be true at the end of the k th execution of the *for* loop.
Suppose the functional dependency considered in the $k + 1$ th execution of the *for* loop is $\beta \rightarrow \gamma$, and that $\beta \subseteq \text{result}$. $\beta \subseteq \text{result}$ implies that $t_1[\beta] = t_2[\beta]$ is true. The facts that $\beta \rightarrow \gamma$ holds for some attribute set R_i in σ , and that $t_1[R_i]$ and $t_2[R_i]$ are in $\Pi_{R_i}(r)$ imply that $t_1[\gamma] = t_2[\gamma]$ is also true. Since γ is now added to *result* by the algorithm, we know that $t_1[\text{result}] = t_2[\text{result}]$ is true at the end of the $k + 1$ th execution of the *for* loop.

Since σ is dependency-preserving and X is a key for R , all attributes in R are in *result* when the algorithm terminates. Thus, $t_1[R] = t_2[R]$ is true, that is, $t_1 = t_2$ – as claimed earlier.

Our claim implies that the size of $\Pi_X(j)$ is equal to the size of j . Note also that $\Pi_X(j) = \Pi_X(r) = r$ (since X is a key for R). Thus we have proved that the size of j equals that of r . Using the result of Practice Exercise 8.12, we know that $r \subseteq j$. Hence we conclude that $r = j$.

Note that since X is trivially in 3NF, $\sigma \cup \{X\}$ is a dependency-preserving lossless-join decomposition into 3NF.

- 8.15** Give an example of a relation schema R' and set F' of functional dependencies such that there are at least three distinct lossless-join decompositions of R' into BCNF.

Answer: Given the relation $R' = (A, B, C, D)$ the set of functional dependencies $F' = A \rightarrow B, C \rightarrow D, B \rightarrow C$ allows three distinct BCNF decompositions.

$$R_1 = \{(A, B), (C, D), (B, C)\}$$

is in BCNF as is

$$R_2 = \{(A, B), (C, D), (A, C)\}$$

$$R_2 = \{(A, B), (C, D), (A, C)\}$$

$$R_3 = \{(B, C), (A, D), (A, B)\}$$

- 8.16** Let a *prime* attribute be one that appears in at least one candidate key. Let α and β be sets of attributes such that $\alpha \rightarrow \beta$ holds, but $\beta \rightarrow \alpha$ does not hold. Let A be an attribute that is not in α , is not in β , and for which $\beta \rightarrow A$ holds. We say that A is *transitively dependent* on α . We can restate our definition of 3NF as follows: A relation schema R is in 3NF with respect to a set F of functional dependencies if there are no nonprime attributes A in R for which A is transitively dependent on a key for R .

Show that this new definition is equivalent to the original one.

Answer: Suppose R is in 3NF according to the textbook definition. We show that it is in 3NF according to the definition in the exercise. Let A be a nonprime attribute in R that is transitively dependent on a key α for R . Then there exists $\beta \subseteq R$ such that $\beta \rightarrow A$, $\alpha \rightarrow \beta$, $A \notin \alpha$, $A \notin \beta$, and $\beta \rightarrow \alpha$ does not hold. But then $\beta \rightarrow A$ violates the textbook definition of 3NF since

- $A \notin \beta$ implies $\beta \rightarrow A$ is nontrivial
- Since $\beta \rightarrow \alpha$ does not hold, β is not a superkey
- A is not any candidate key, since A is nonprime

Now we show that if R is in 3NF according to the exercise definition, it is in 3NF according to the textbook definition. Suppose R is not in 3NF according to the textbook definition. Then there is an FD $\alpha \rightarrow \beta$ that fails all three conditions. Thus

- $\alpha \rightarrow \beta$ is nontrivial.
- α is not a superkey for R .
- Some A in $\beta - \alpha$ is not in any candidate key.

This implies that A is nonprime and $\alpha \rightarrow A$. Let γ be a candidate key for R . Then $\gamma \rightarrow \alpha$, $\alpha \rightarrow \gamma$ does not hold (since α is not a superkey), $A \notin \alpha$, and $A \notin \gamma$ (since A is nonprime). Thus A is transitively dependent on γ , violating the exercise definition.

- 8.17 A functional dependency $\alpha \rightarrow \beta$ is called a **partial dependency** if there is a proper subset γ of α such that $\gamma \rightarrow \beta$. We say that β is *partially dependent* on α . A relation schema R is in **second normal form** (2NF) if each attribute A in R meets one of the following criteria:

- It appears in a candidate key.
- It is not partially dependent on a candidate key.

Show that every 3NF schema is in 2NF. (*Hint: Show that every partial dependency is a transitive dependency.*)

Answer: Referring to the definitions in Practice Exercise 8.16, a relation schema R is said to be in 3NF if there is no non-prime attribute A in R for which A is transitively dependent on a key for R .

We can also rewrite the definition of 2NF given here as :

“A relation schema R is in 2NF if no non-prime attribute A is partially dependent on any candidate key for R .”

To prove that every 3NF schema is in 2NF, it suffices to show that if a non-prime attribute A is partially dependent on a candidate key α , then A is also transitively dependent on the key α .

Let A be a non-prime attribute in R . Let α be a candidate key for R . Suppose A is partially dependent on α .

- From the definition of a partial dependency, we know that for some proper subset β of α , $\beta \rightarrow A$.
- Since $\beta \subset \alpha$, $\alpha \rightarrow \beta$. Also, $\beta \rightarrow \alpha$ does not hold, since α is a candidate key.
- Finally, since A is non-prime, it cannot be in either β or α .

Thus we conclude that $\alpha \rightarrow A$ is a transitive dependency. Hence we have proved that every 3NF schema is also in 2NF.

- 8.18 Give an example of a relation schema R and a set of dependencies such that R is in BCNF but is not in 4NF.

Answer:

$$\begin{array}{l} R(A, B, C) \\ A \twoheadrightarrow B \end{array}$$

试用水印

```

result :=  $\emptyset$ ;
/* fdcount is an array whose ith element contains the number
   of attributes on the left side of the ith FD that are
   not yet known to be in  $\alpha^+$  */
for i := 1 to |F| do
  begin
    let  $\beta \rightarrow \gamma$  denote the ith FD;
    fdcount [i] := | $\beta$ |;
  end
/* appears is an array with one entry for each attribute. The
   entry for attribute A is a list of integers. Each integer
   i on the list indicates that A appears on the left side
   of the ith FD */
for each attribute A do
  begin
    appears [A] := NIL;
    for i := 1 to |F| do
      begin
        let  $\beta \rightarrow \gamma$  denote the ith FD;
        if  $A \in \beta$  then add i to appears [A];
      end
    end
  addin ( $\alpha$ );
  return (result);

procedure addin ( $\alpha$ );
for each attribute A in  $\alpha$  do
  begin
    if  $A \notin \text{result}$  then
      begin
        result := result  $\cup$  {A};
        for each element i of appears [A] do
          begin
            fdcount [i] := fdcount [i] - 1;
            if fdcount [i] := 0 then
              begin
                let  $\beta \rightarrow \gamma$  denote the ith FD;
                addin ( $\gamma$ );
              end
            end
          end
        end
      end
    end
  end
end

```

Figure 8.18. An algorithm to compute α^+ .

CHAPTER 8



Relational Database Design

This chapter presents the principles of relational database design. Undergraduates frequently find this chapter difficult. It is acceptable to cover only Sections 8.1 and 8.3 for classes that find the material particularly difficult. However, a careful study of data dependencies and normalization is a good way to introduce students to the formal aspects of relational database theory.

There are many ways of stating the definitions of the normal forms. We have chosen a style which we think is the easiest to present and which most clearly conveys the intuition of the normal forms.

Exercises

- 8.19 Give a lossless-join decomposition into BCNF of schema R of Exercise 8.1.
Answer: From Exercise 8.6, we know that $B \rightarrow D$ is nontrivial and the left hand side is not a superkey. By the algorithm of Figure 8.11 we derive the relations $\{(A, B, C, E), (B, D)\}$. This is in BCNF.
- 8.20 Give a lossless-join, dependency-preserving decomposition into 3NF of schema R of Practice Exercise 8.1.
Answer: First we note that the dependencies given in Practice Exercise 8.1 form a canonical cover. Generating the schema from the algorithm of Figure 8.12 we get

$$R' = \{(A, B, C), (C, D, E), (B, D), (E, A)\}.$$

Schema (A, B, C) contains a candidate key. Therefore R' is a third normal form dependency-preserving lossless-join decomposition.

Note that the original schema $R = (A, B, C, D, E)$ is already in 3NF. Thus, it was not necessary to apply the algorithm as we have done above. The single original schema is trivially a lossless join, dependency-preserving decomposition.

8.21 Normalize the following schema, with given constraints, to 4NF.

books(*accessionno*, *isbn*, *title*, *author*, *publisher*)
users(*userid*, *name*, *deptid*, *deptname*)
 $accessionno \rightarrow isbn$
 $isbn \rightarrow title$
 $isbn \rightarrow publisher$
 $isbn \twoheadrightarrow author$
 $userid \rightarrow name$
 $userid \rightarrow deptid$
 $deptid \rightarrow deptname$

Answer: In *books*, we see that

$$isbn \twoheadrightarrow title, publisher, author$$

and yet, *isbn* is not a super key. Thus, we break *books* into

books_accnno(*accessionno*, *isbn*)
books_details(*isbn*, *title*, *publisher*, *author*)

After this, we still have

$$isbn \twoheadrightarrow author$$

but neither is *isbn* a primary key of *book_details*, nor are the attributes of *book_details* equal to $\{isbn\} \cup \{author\}$. Therefore we decompose *book_details* again into

books_details1(*isbn*, *title*, *publisher*)
books_authors(*isbn*, *author*)

Similarly, in *users*,

$$deptid \rightarrow deptname$$

and yet, *deptid* is not a super key. Hence, we break *users* to

users(*userid*, *name*, *deptid*)
departments(*deptid*, *deptname*)

We verify that there are no further functional or multivalued dependencies that cause violation of 4NF, so the final set of relations are:

```
books_accno(accessionno, isbn)
books_details1(isbn, title, publisher)
books_authors(isbn, author) users(userid, name, deptid)
departments(deptid, deptname)
```

- 8.22 Explain what is meant by *repetition of information* and *inability to represent information*. Explain why each of these properties may indicate a bad relational-database design.

Answer:

- Repetition of information is a condition in a relational database where the values of one attribute are determined by the values of another attribute in the same relation, and both values are repeated throughout the relation. This is a bad relational database design because it increases the storage required for the relation and it makes updating the relation more difficult, and can lead to inconsistent data if updates are done to one copy of the value, but not to another.
- Inability to represent information is a condition where a relationship exists among only a proper subset of the attributes in a relation. This is bad relational database design because all the unrelated attributes must be filled with null values otherwise a tuple without the unrelated information cannot be inserted into the relation.
- Inability to represent information can also occur because of loss of information which results from the decomposition of one relation into two relations, which cannot be combined to recreate the original relation. Such a lossy decomposition may happen implicitly, even without explicitly carrying out decomposition, if the initial relational schema itself corresponds to the decomposition.

- 8.23 Why are certain functional dependencies called *trivial* functional dependencies?

Answer: Certain functional dependencies are called trivial functional dependencies because they are satisfied by all relations.

- 8.24 Use the definition of functional dependency to argue that each of Armstrong's axioms (reflexivity, augmentation, and transitivity) is sound.

Answer: The definition of functional dependency is: $\alpha \rightarrow \beta$ holds on R if in any legal relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, it is also the case that $t_1[\beta] = t_2[\beta]$.

Reflexivity rule: if α is a set of attributes, and $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$.
 Assume $\exists t_1$ and t_2 such that $t_1[\alpha] = t_2[\alpha]$

$$\begin{array}{ll} t_1[\beta] = t_2[\beta] & \text{since } \beta \subseteq \alpha \\ \alpha \rightarrow \beta & \text{definition of FD} \end{array}$$

Augmentation rule: if $\alpha \rightarrow \beta$, and γ is a set of attributes, then $\gamma \alpha \rightarrow \gamma \beta$.
 Assume $\exists t_1, t_2$ such that $t_1[\gamma \alpha] = t_2[\gamma \alpha]$

$$\begin{array}{ll} t_1[\gamma] = t_2[\gamma] & \gamma \subseteq \gamma \alpha \\ t_1[\alpha] = t_2[\alpha] & \alpha \subseteq \gamma \alpha \\ t_1[\beta] = t_2[\beta] & \text{definition of } \alpha \rightarrow \beta \\ t_1[\gamma \beta] = t_2[\gamma \beta] & \gamma \beta = \gamma \cup \beta \\ \gamma \alpha \rightarrow \gamma \beta & \text{definition of FD} \end{array}$$

Transitivity rule: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$.
 Assume $\exists t_1, t_2$ such that $t_1[\alpha] = t_2[\alpha]$

$$\begin{array}{ll} t_1[\beta] = t_2[\beta] & \text{definition of } \alpha \rightarrow \beta \\ t_1[\gamma] = t_2[\gamma] & \text{definition of } \beta \rightarrow \gamma \\ \alpha \rightarrow \gamma & \text{definition of FD} \end{array}$$

8.25 Consider the following proposed rule for functional dependencies: If $\alpha \rightarrow \beta$ and $\gamma \rightarrow \beta$, then $\alpha \rightarrow \gamma$. Prove that this rule is *not* sound by showing a relation r that satisfies $\alpha \rightarrow \beta$ and $\gamma \rightarrow \beta$, but does not satisfy $\alpha \rightarrow \gamma$.

Answer: Consider the following rule: if $A \rightarrow B$ and $C \rightarrow B$, then $A \rightarrow C$. That is, $\alpha = A, \beta = B, \gamma = C$. The following relation r is a counterexample to the rule.

r :

A	B	C
a_1	b_1	c_1
a_1	b_1	c_2

Note: $A \rightarrow B$ and $C \rightarrow B$, (since no 2 tuples have the same C value, $C \rightarrow B$ is true trivially). However, it is not the case that $A \rightarrow C$ since the same A value is in two tuples, but the C value in those tuples disagree.

8.26 Use Armstrong's axioms to prove the soundness of the decomposition rule.

Answer: The decomposition rule, and its derivation from Armstrong's axioms are given below:

if $\alpha \rightarrow \beta\gamma$, then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$.

$$\begin{array}{ll} \alpha \rightarrow \beta\gamma & \text{given} \\ \beta\gamma \rightarrow \beta & \text{reflexivity rule} \\ \alpha \rightarrow \beta & \text{transitivity rule} \\ \beta\gamma \rightarrow \gamma & \text{reflexive rule} \\ \alpha \rightarrow \gamma & \text{transitive rule} \end{array}$$

8.27 Using the functional dependencies of Practice Exercise 8.6, compute B^+ .

Answer: Computing B^+ by the algorithm in Figure 8.8 we start with $result = \{B\}$. Considering FDs of the form $\beta \rightarrow \gamma$ in F , we find that the only dependencies satisfying $\beta \subseteq result$ are $B \rightarrow B$ and $B \rightarrow D$. Therefore $result = \{B, D\}$. No more dependencies in F apply now. Therefore $B^+ = \{B, D\}$

- 8.28 Show that the following decomposition of the schema R of Practice Exercise 8.1 is not a lossless-join decomposition:

(A, B, C)
 $(C, D, E).$

Hint: Give an example of a relation r on schema R such that

$$\Pi_{A, B, C}(r) \bowtie \Pi_{C, D, E}(r) \neq r$$

Answer: Following the hint, use the following example of r :

A	B	C	D	E
a_1	b_1	c_1	d_1	e_1
a_2	b_2	c_1	d_2	e_2

With $R_1 = (A, B, C)$, $R_2 = (C, D, E)$:

- a. $\Pi_{R_1}(r)$ would be:

A	B	C
a_1	b_1	c_1
a_2	b_2	c_1

- b. $\Pi_{R_2}(r)$ would be:

C	D	E
c_1	d_1	e_1
c_1	d_2	e_2

- c. $\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$ would be:

A	B	C	D	E
a_1	b_1	c_1	d_1	e_1
a_1	b_1	c_1	d_2	e_2
a_2	b_2	c_1	d_1	e_1
a_2	b_2	c_1	d_2	e_2

Clearly, $\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \neq r$. Therefore, this is a lossy join.

8.29 Consider the following set F of functional dependencies on the relation schema $r(A, B, C, D, E, F)$:

$$\begin{aligned} A &\rightarrow BCD \\ BC &\rightarrow DE \\ B &\rightarrow D \\ D &\rightarrow A \end{aligned}$$

- Compute B^+ .
- Prove (using Armstrong's axioms) that AF is a superkey.
- Compute a canonical cover for the above set of functional dependencies F ; give each step of your derivation with an explanation.
- Give a 3NF decomposition of r based on the canonical cover.
- Give a BCNF decomposition of r using the original set of functional dependencies.
- Can you get the same BCNF decomposition of r as above, using the canonical cover?

Answer:

- $B \rightarrow BD$ (third dependency)
 $BD \rightarrow ABD$ (fourth dependency)
 $ABD \rightarrow ABCD$ (first dependency)
 $ABCD \rightarrow ABCDE$ (second dependency)

Thus, $B^+ = ABCDE$

- Prove (using Armstrong's axioms) that AF is a superkey.

$$\begin{aligned} A &\rightarrow BCD \text{ (Given)} \\ A &\rightarrow ABCD \text{ (Augmentation with A)} \\ BC &\rightarrow DE \text{ (Given)} \\ ABCD &\rightarrow ABCDE \text{ (Augmentation with ABCD)} \\ A &\rightarrow ABCDE \text{ (Transitivity)} \\ AF &\rightarrow ABCDEF \text{ (Augmentation with F)} \end{aligned}$$

- We see that D is extraneous in dep. 1 and 2, because of dep. 3. Removing these two, we get the new set of rules

$$\begin{aligned} A &\rightarrow BC \\ BC &\rightarrow E \\ B &\rightarrow D \\ D &\rightarrow A \end{aligned}$$

Now notice that B^+ is $ABCDE$, and in particular, the FD $B \rightarrow E$ can be determined from this set. Thus, the attribute C is extraneous in

the third dependency. Removing this attribute, and combining with the third FD, we get the final canonical cover as :

$$\begin{aligned} A &\rightarrow BC \\ B &\rightarrow DE \\ D &\rightarrow A \end{aligned}$$

Here, no attribute is extraneous in any FD.

- d. We see that there is no FD in the canonical cover such that the set of attributes is a subset of any other FD in the canonical cover. Thus, each FD gives rise to its own relation, giving

$$\begin{aligned} r_1(A, B, C) \\ r_2(B, D, E) \\ r_3(D, A) \end{aligned}$$

Now the attribute F is not dependent on any attribute. Thus, it must be a part of every superkey. Also, none of the relations in the above schema have F , and hence, none of them have a superkey. Thus, we need to add a new relation with a superkey.

$$r_4(A, F)$$

- e. We start with

$$r(A, B, C, D, E, F)$$

We see that the relation is not in BCNF because of the first FD. Hence, we decompose it accordingly to get

$$r_1(A, B, C, D) \ r_2(A, E, F)$$

Now we notice that $A \rightarrow E$ is an FD in F^+ , and causes r_2 to violate BCNF. Once again, decomposing r_2 gives

$$r_1(A, B, C, D) \ r_2(A, F) \ r_3(A, E)$$

This schema is now in BCNF.

- f. Can you get the same BCNF decomposition of r as above, using the canonical cover?

If we use the functional dependencies in the preceding canonical cover directly, we cannot get the above decomposition. However, we can infer the original dependencies from the canonical cover, and if we use those for BCNF decomposition, we would be able to get the same decomposition.

- 8.30 List the three design goals for relational databases, and explain why each is desirable.

Answer: The three design goals are lossless-join decompositions, dependency preserving decompositions, and minimization of repetition of

information. They are desirable so we can maintain an accurate database, check correctness of updates quickly, and use the smallest amount of space possible.

- 8.31 In designing a relational database, why might we choose a non-BCNF design?

Answer: BCNF is not always dependency preserving. Therefore, we may want to choose another normal form (specifically, 3NF) in order to make checking dependencies easier during updates. This would avoid joins to check dependencies and increase system performance.

- 8.32 Given the three goals of relational-database design, is there any reason to design a database schema that is in 2NF, but is in no higher-order normal form? (See Practice Exercise 8.17 for the definition of 2NF.)

Answer: The three design goals of relational databases are to avoid

- Repetition of information
- Inability to represent information
- Loss of information.

2NF does not prohibit as much repetition of information since the schema (A, B, C) with dependencies $A \rightarrow B$ and $B \rightarrow C$ is allowed under 2NF, although the same (B, C) pair could be associated with many A values, needlessly duplicating C values. To avoid this we must go to 3NF. Repetition of information is allowed in 3NF in some but not all of the cases where it is allowed in 2NF. Thus, in general, 3NF reduces repetition of information. Since we can always achieve a lossless join 3NF decomposition, there is no loss of information needed in going from 2NF to 3NF.

Note that the decomposition $\{(A, B), (B, C)\}$ is a dependency-preserving and lossless-join 3NF decomposition of the schema (A, B, C) . However, in case we choose this decomposition, retrieving information about the relationship between A, B and C requires a join of two relations, which is avoided in the corresponding 2NF decomposition.

Thus, the decision of which normal form to choose depends upon how the cost of dependency checking compares with the cost of the joins. Usually, the 3NF would be preferred. Dependency checks need to be made with *every* insert or update to the instances of a 2NF schema, whereas, only some queries will require the join of instances of a 3NF schema.

- 8.33 Given a relational schema $r(A, B, C, D)$, does $A \twoheadrightarrow BC$ logically imply $A \twoheadrightarrow B$ and $A \twoheadrightarrow C$? If yes prove it, else give a counter example.

Answer: $A \twoheadrightarrow BC$ holds on the following table:

$r :$

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_2	d_2
a_1	b_1	c_1	d_2
a_1	b_2	c_2	d_1

If $A \twoheadrightarrow B$, then we know that there exists t_1 and t_3 such that $t_1[B] = t_3[B]$. Thus, we must choose one of the following for t_1 and t_3 :

- $t_1 = r_1$ and $t_3 = r_3$, or $t_1 = r_3$ and $t_3 = r_1$:
Choosing either $t_2 = r_2$ or $t_2 = r_4$, $t_3[C] \neq t_2[C]$.
- $t_1 = r_2$ and $t_3 = r_4$, or $t_1 = r_4$ and $t_3 = r_2$:
Choosing either $t_2 = r_1$ or $t_2 = r_3$, $t_3[C] \neq t_2[C]$.

Therefore, the condition $t_3[C] = t_2[C]$ can not be satisfied, so the conjecture is false.

8.34 Explain why 4NF is a normal form more desirable than BCNF.

Answer: 4NF is more desirable than BCNF because it reduces the repetition of information. If we consider a BCNF schema not in 4NF (see Practice Exercise 7.16), we observe that decomposition into 4NF does not lose information provided that a lossless join decomposition is used, yet redundancy is reduced.